

Chapter 9:

Indirect Volume Visualization

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

June 10/16, 2026

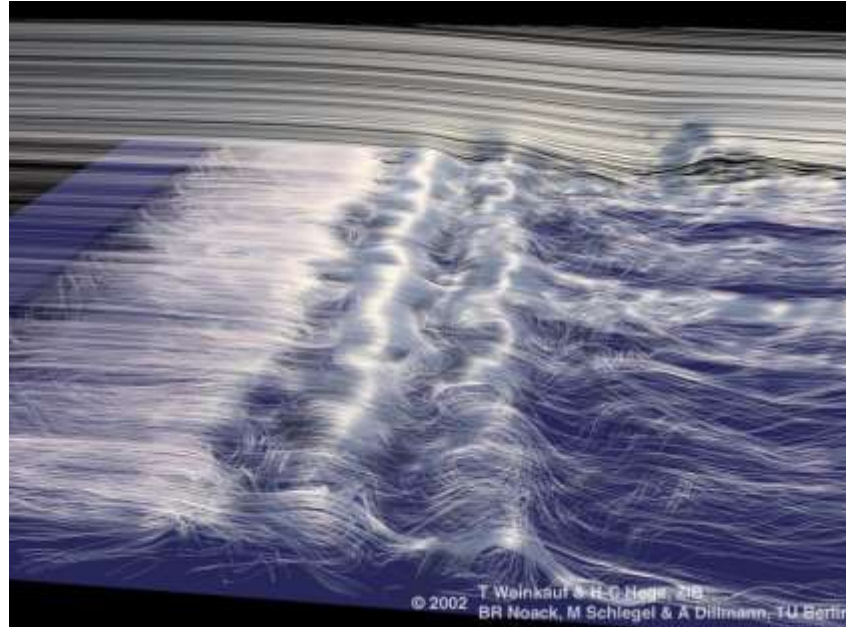
Section 9.1:

Introduction to Volume Visualization

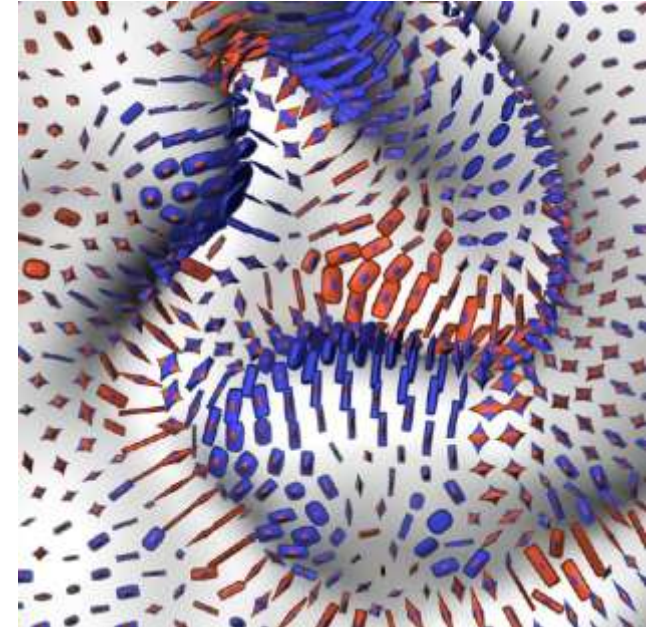
Where We Are In Scientific Visualization



Scalar Fields



Vector Fields



Tensor Fields

$$f : D \rightarrow \mathbb{R} \quad \text{with} \quad D \subseteq \mathbb{R}^3$$

Volumetric Data in Medical Imaging



Computed
Tomography



Fused CT and
Magnetic
Resonance
Imaging (MRI)

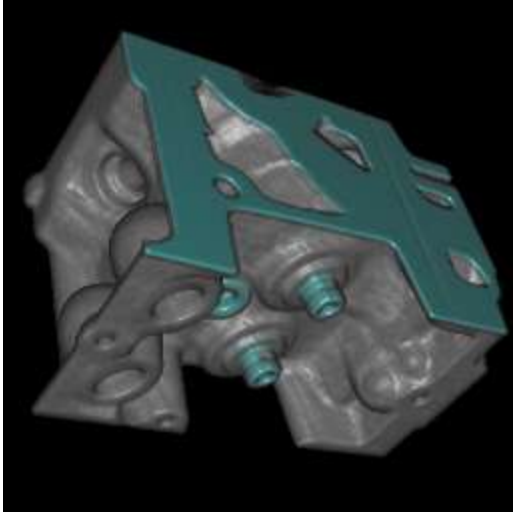


Fused CT and
Positron Emission
Tomography (PET)

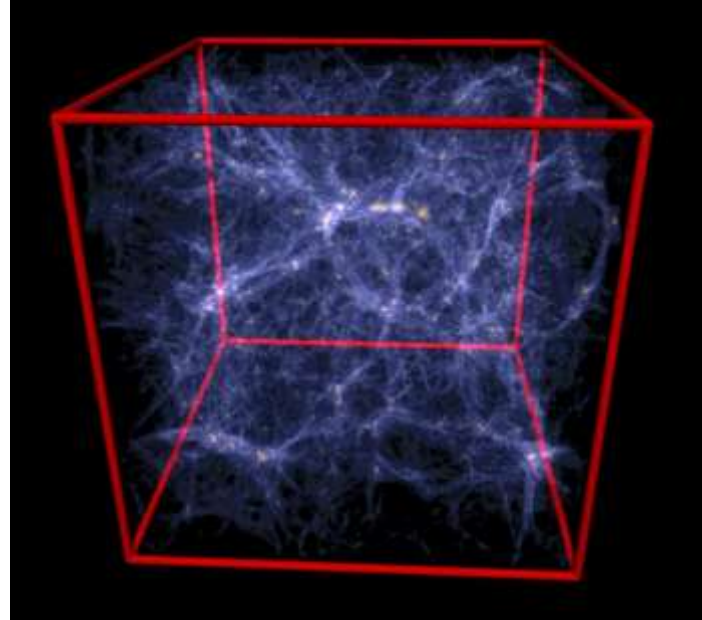


3D Ultrasound

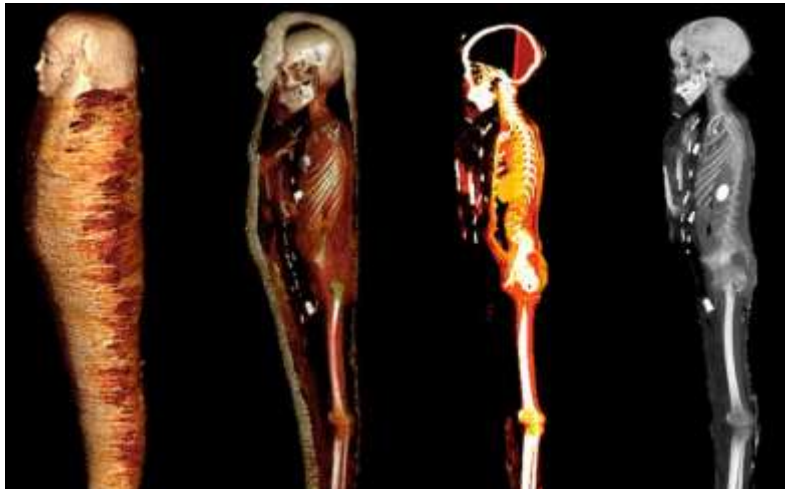
Volumetric Data in Science/Engineering



Industrial CT



Numerical Simulation:
Formation of
Cosmological
Structures



Scientific CT



Numerical Simulation:
Fuel injection

Strategies for Volume Visualization

1. **Slicing** (Section 8.6)

- Visualize color coded 2D cuts through the volume.

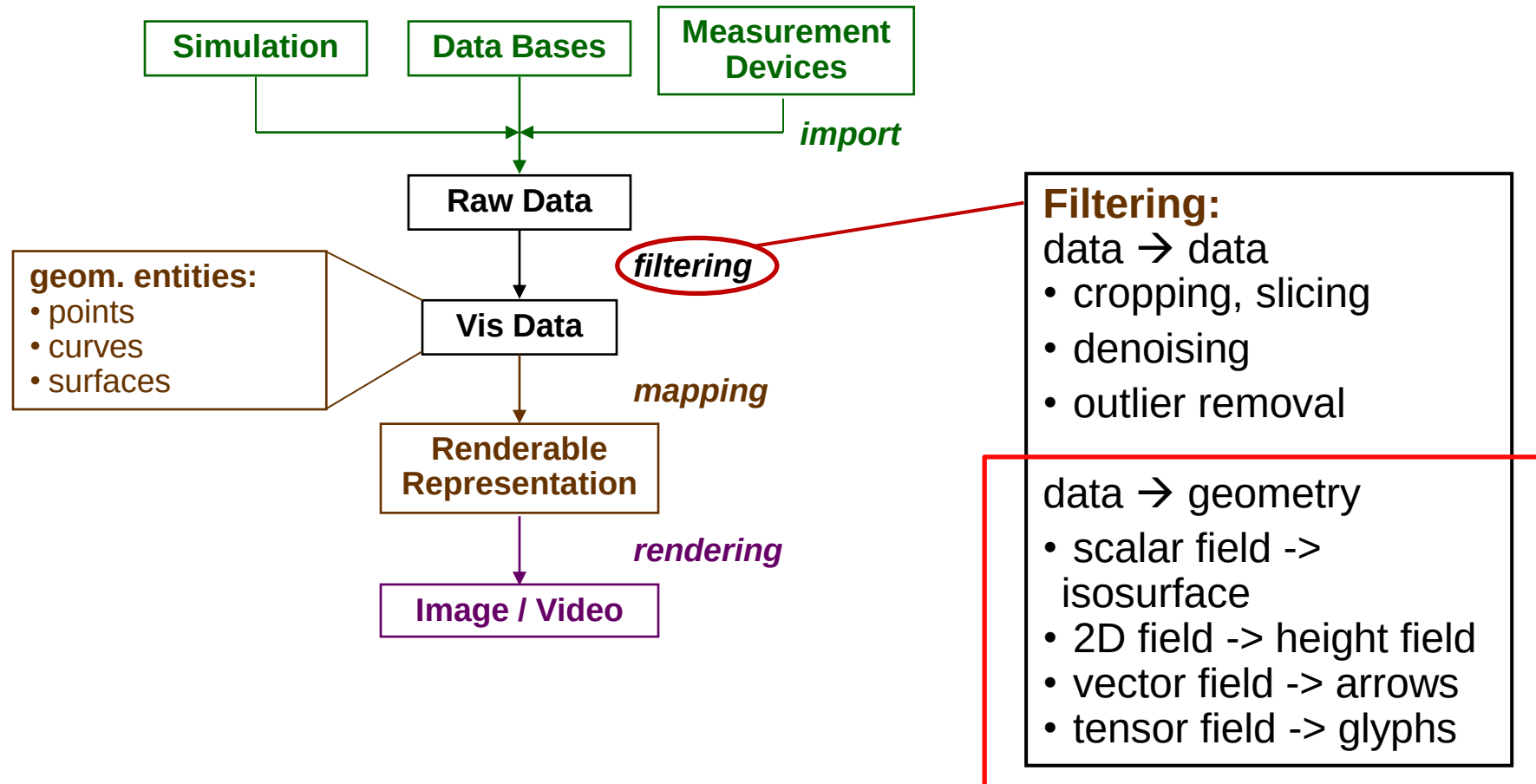
2. **Indirect volume rendering** (this chapter)

- Extract surfaces from the volume, which usually represent objects in the volume and can be rendered with traditional techniques.

3. **Direct volume rendering** (next chapter)

- Directly assign color and opacity to each 3D point, based on its data. Create an image by simulating light transport according to the laws of physics.

Where We Are In The Vis Pipeline



Counts as filtering in VTK; conceptually, is often counted as mapping

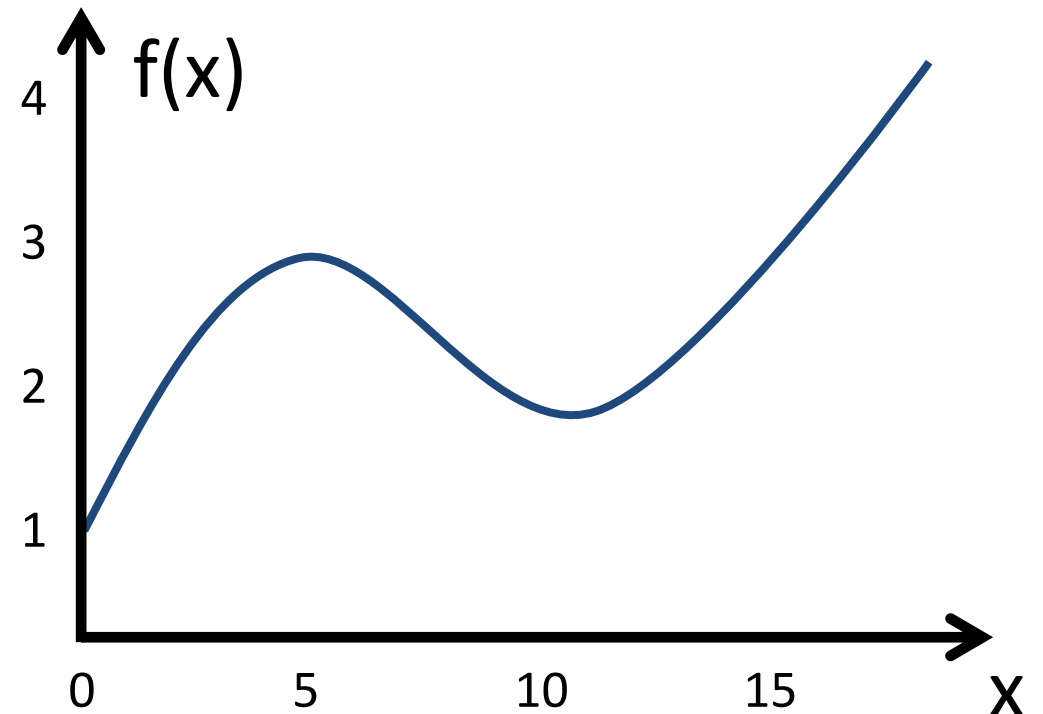
Section 9.2: Isocurves

Level Sets in 1D Functions

Standard visualization of a “1D Scalar Field”:

$$f: \Omega \subset \mathbb{R} \rightarrow \mathbb{R}$$

- **Function plot** consists of points $\{(x, f(x)) | x \in \mathbb{R}\}$
 - 1D manifold (curve)
- **Level set** $\{x | f(x) = c\}$ for a given isovalue $c \in \mathbb{R}$ typically contains isolated points

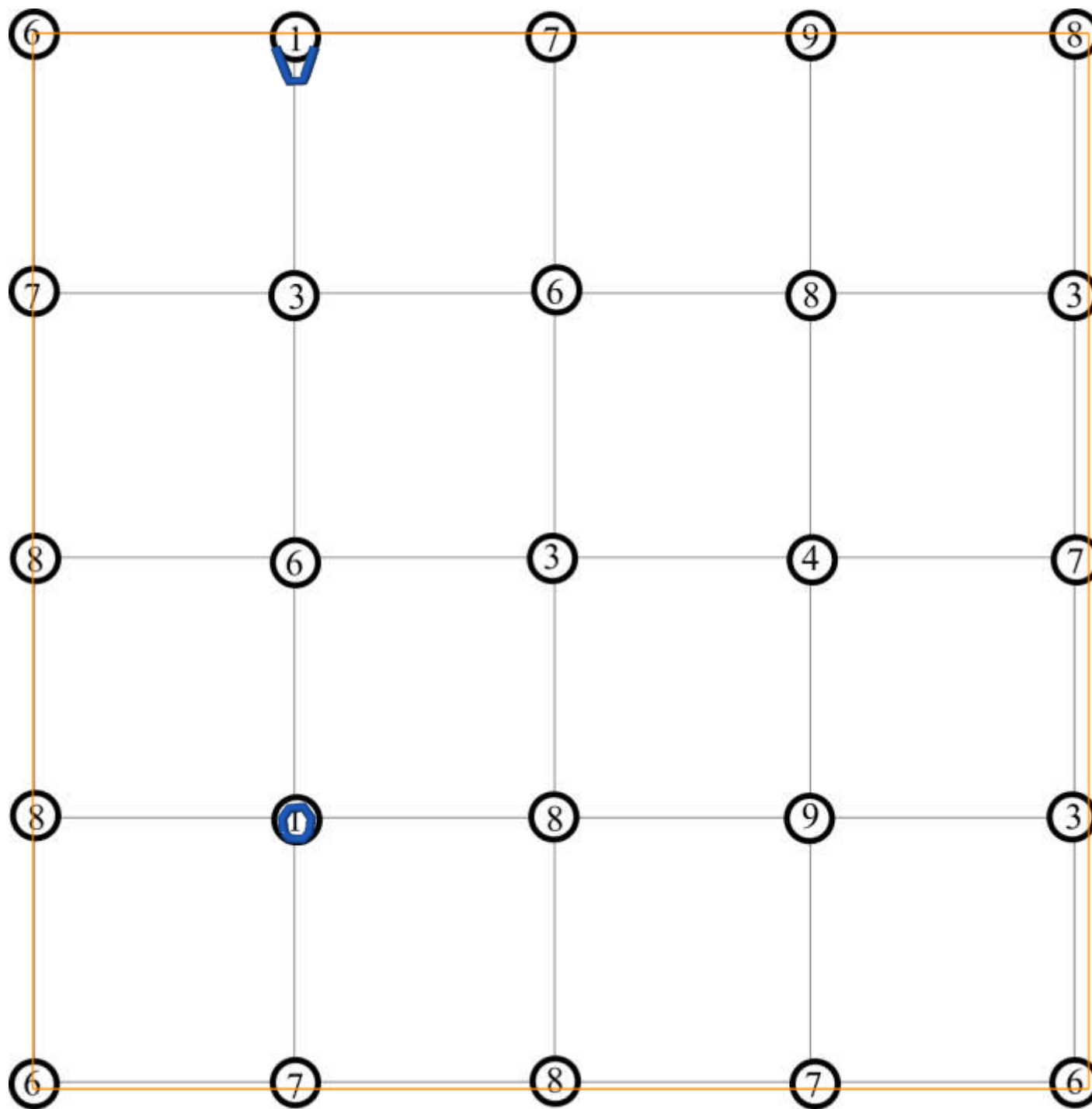


Isocurves in 2D Scalar Fields

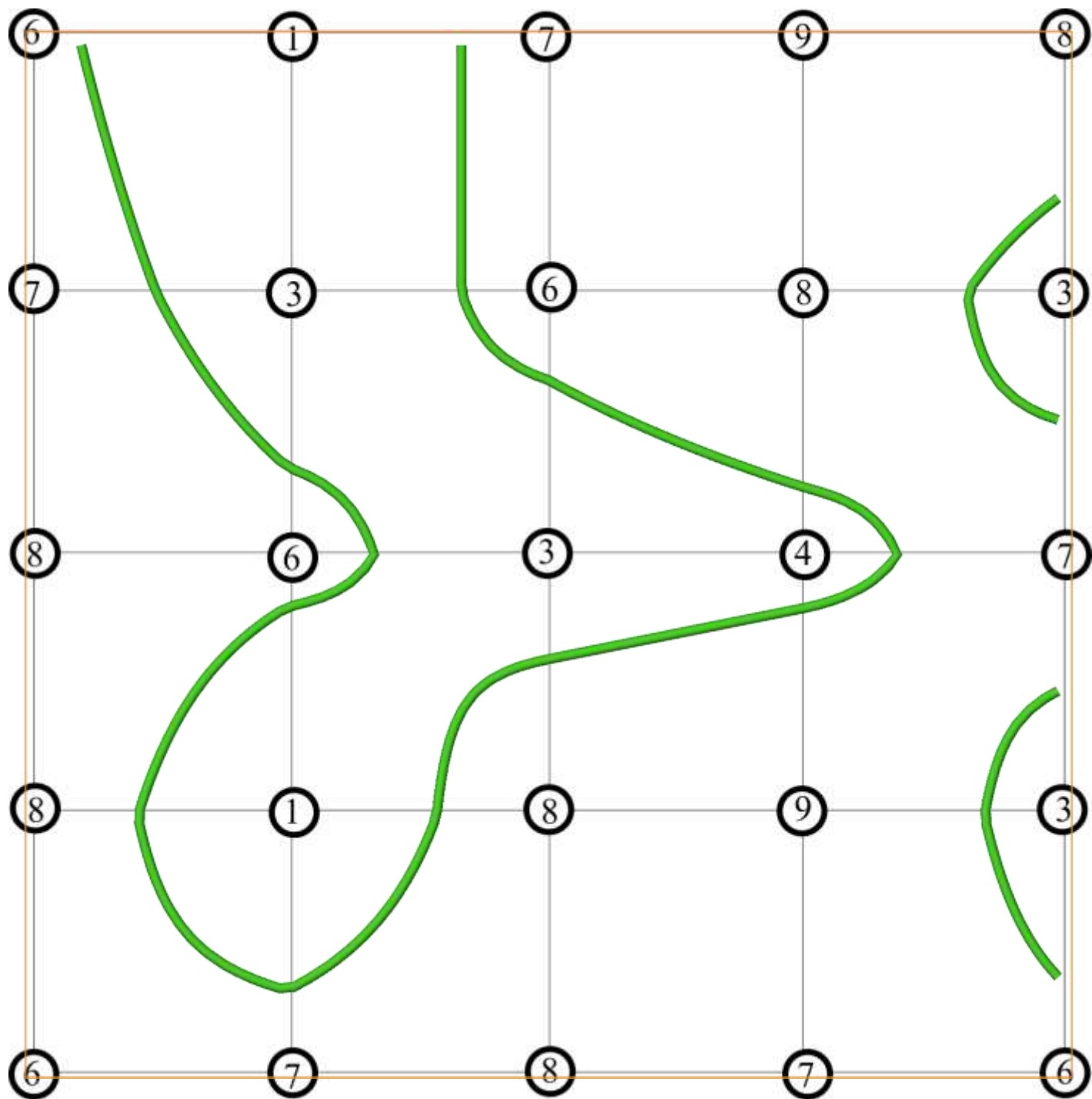
- 2D scalar fields $f: \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}$ can be plotted as a height field
 - Points $\{(x, y, f(x, y)) | (x, y) \in \mathbb{R}^2\}$ form a 2D manifold (surface)
- If f is differentiable and $\nabla f \neq \mathbf{0}$, the level sets $\{(x, y) | f(x, y) = c\}$ for isovalue $c \in \mathbb{R}$ are curves (isocurves)
 - Contour lines often provide outlines of important objects



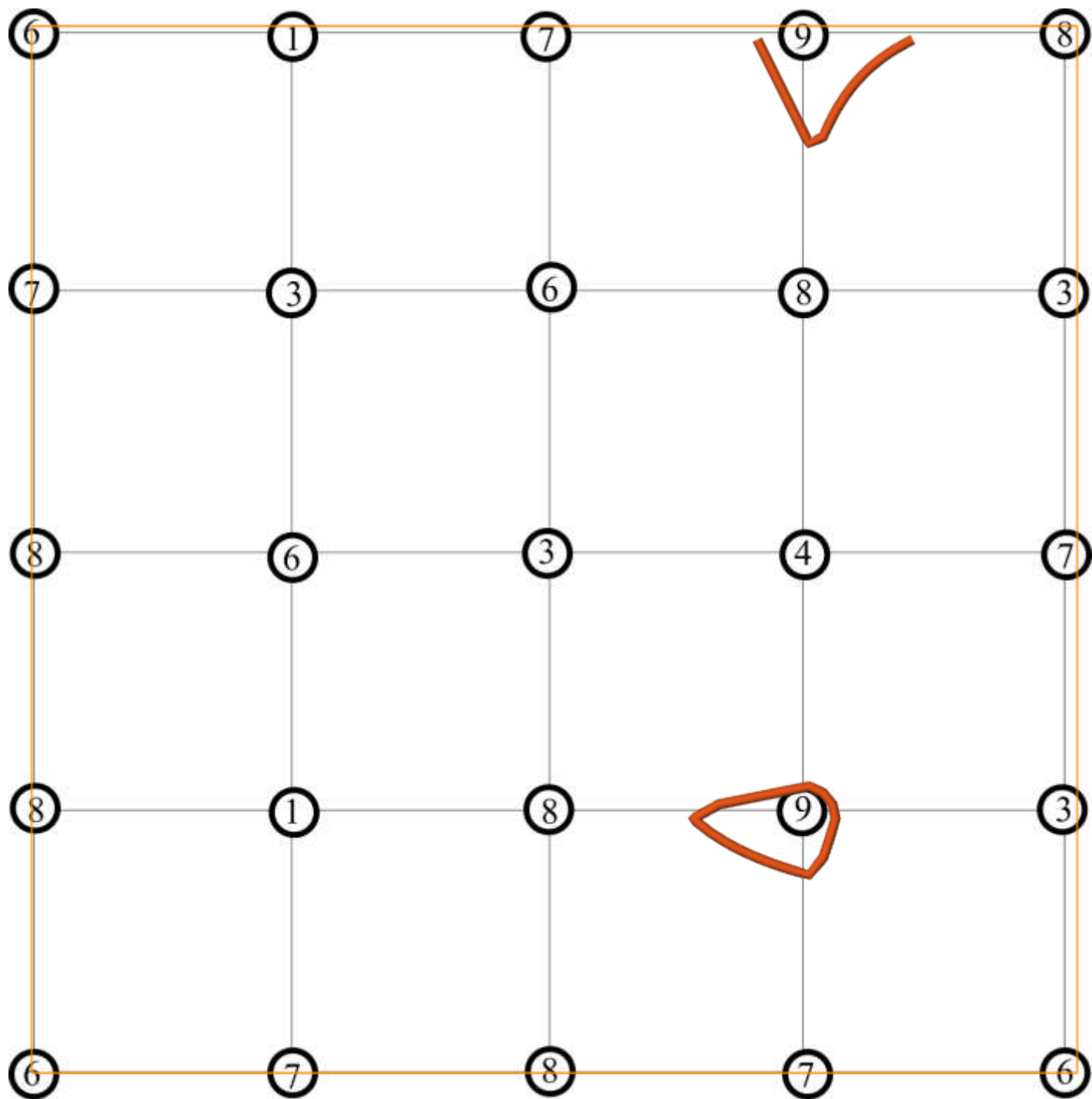
$c = 1.5$



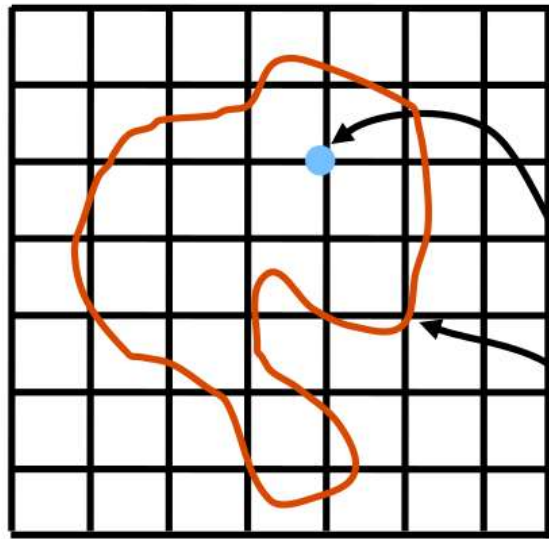
$c = 5$



$c = 8.5$



Interpolation vs. Approximation



Data is given at *discrete* points $f(x,y,z)$

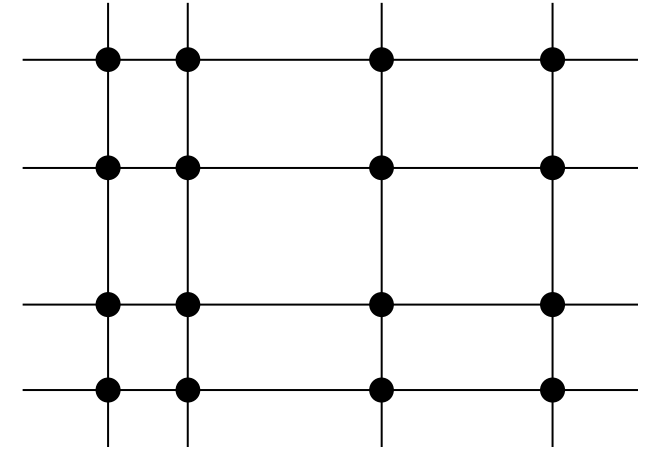
We would like to reconstruct a *continuous* isocurve

Two basic strategies:

- **Interpolate** to get the „true“ answer
 - Subsampling, ray casting (chapter 10), Newton's method
 - *Result*: A (dense) set of points
- **Approximate** to get a „good“ answer
 - Geometric approximation of part of isocurve within each cell
 - *Result*: A set of line segments (2D) or triangles (3D)

Properties of Marching Squares Algorithm

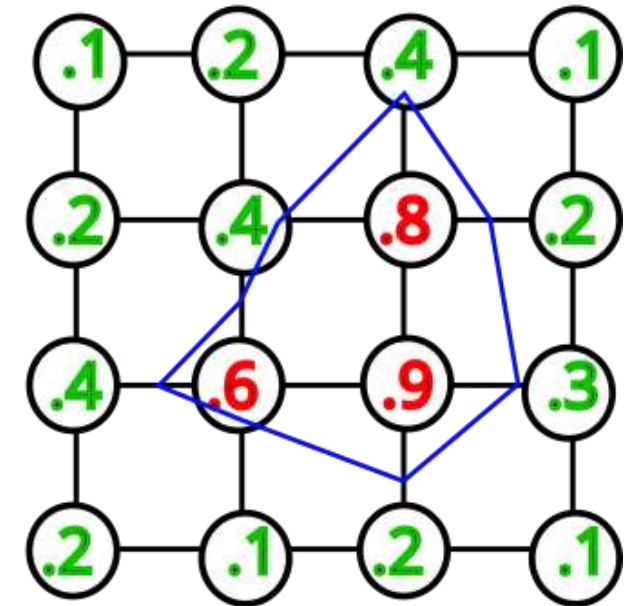
- Input:
 - Rectilinear grid
 - Scalar value f_{ij} at each vertex
- Properties of the algorithm:
 - Divide and conquer: considers cells independently from each other
 - Guarantees never to miss isocurves (assuming bilinear interpolation)
 - Produces the same connectivity as bilinear interpolation within cells



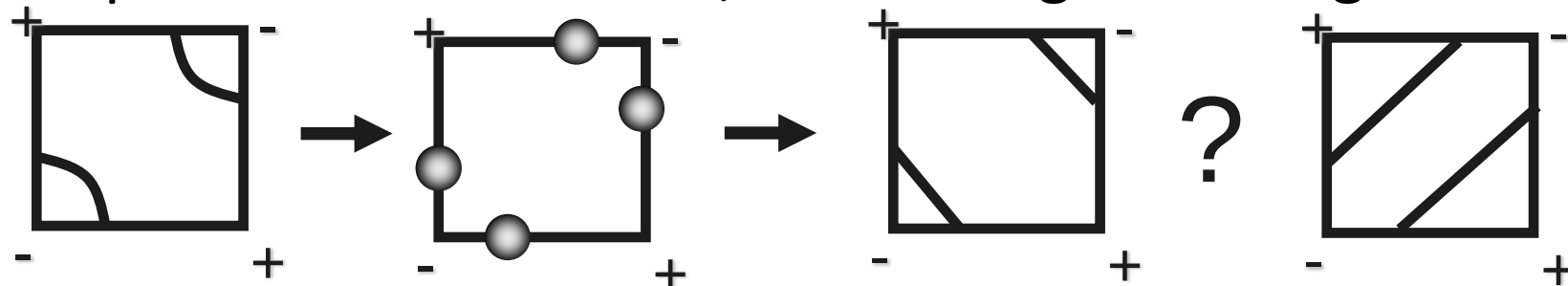
Basic Steps of Marching Squares

- The **Marching Squares** algorithm computes contour (as a set of polygons) from pixel values and threshold c

1. Classify each pixel as **inside** or **outside**
2. Interpolate the point of intersection as relative distance α between **inside** and **outside**

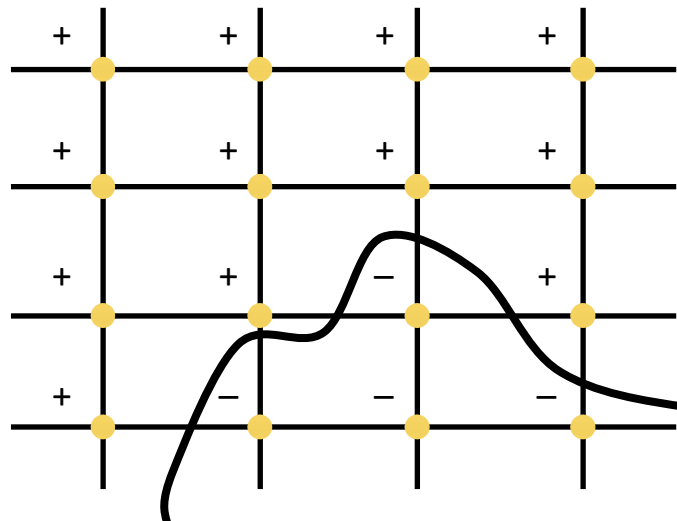


3. Connect points of intersection, accounting for ambiguities



Marching Squares: Finding Cells

- Which cells will be intersected ?
 - Initially mark all vertices by + or − ,
depending on the conditions $f_{ij} > c$, $f_{ij} \leq c$
- No isocurve passes through cells (=rectangles) which have the same sign at all four vertices
 - Only have to deal with edges with different signs



Marching Squares: Finding Intersection Points

- Summary of previous slide:

1. Compute $f_{min} = \min(f_{i,j}, f_{i+1,j}, f_{i,j+1}, f_{i+1,j+1})$
 $f_{max} = \max(f_{i,j}, f_{i+1,j}, f_{i,j+1}, f_{i+1,j+1})$

2. For isovalue c , find all grid cells with

$$f_{min} \leq c \leq f_{max}$$

- To find points of intersection along boundary:
 - Linear interpolation along the cell edges. For inside value v_i , outside value v_o , isovalue c , the relative position along the edge is

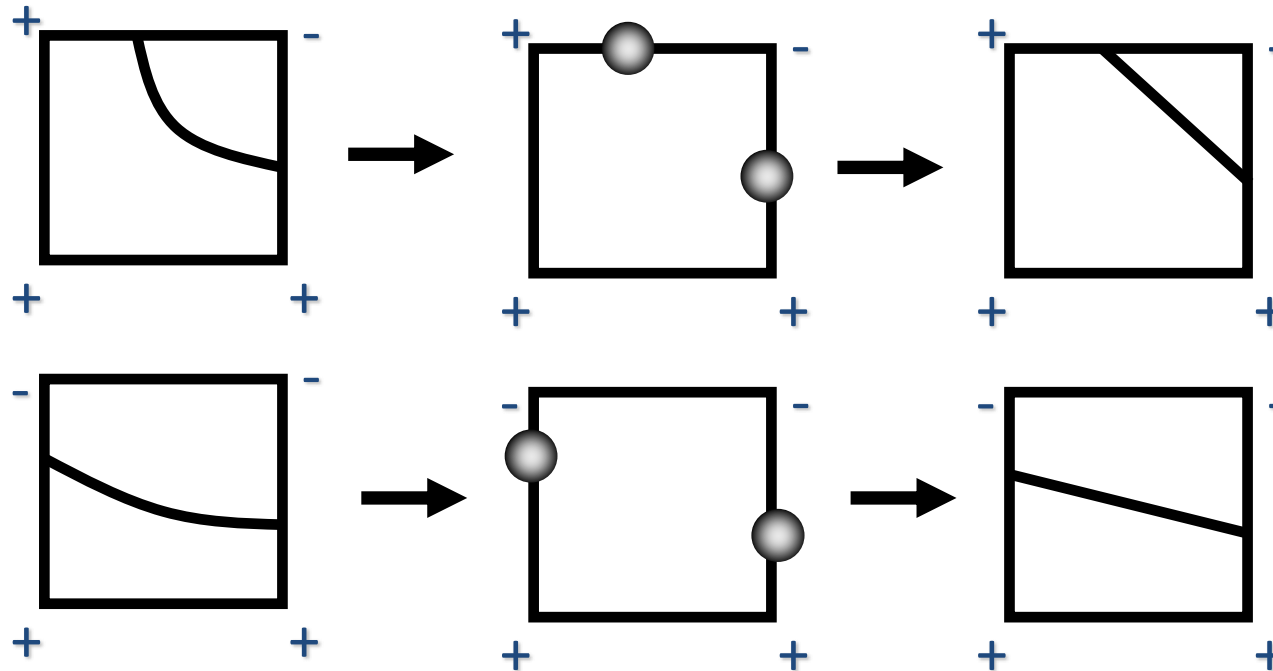
$$\alpha = \frac{c - v_i}{v_o - v_i}$$

Symbolic Perturbation

- It **simplifies** our algorithm to assume that isocontours will never run exactly through one of the vertices
 - **Theoretical Justification:** If isovalue or node values are taken from continuous distributions, the probability of them being exactly equal is zero
 - **Practical Implementation:** If c equals a node value f_{ij} , perturb f_{ij} to $f_{ij}-\varepsilon$ with an infinitesimal ε
 - Rather than picking a particular ε , we do this **symbolically** by counting $f_{ij} \leq c$ as - in the classification

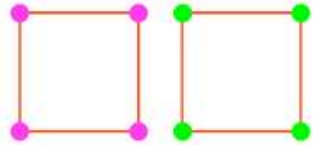
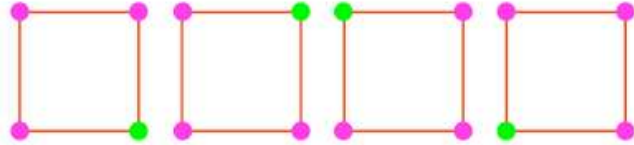
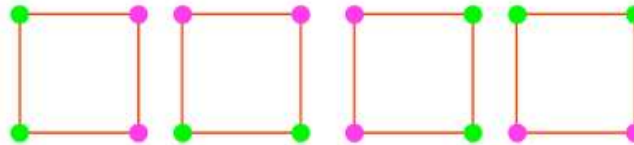
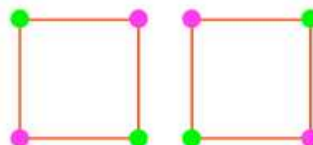
Marching Squares: Connecting the Dots

- Connect points of intersection with line segments
- Solution is obvious when there are only two points
- General rule to connect the points of intersection?



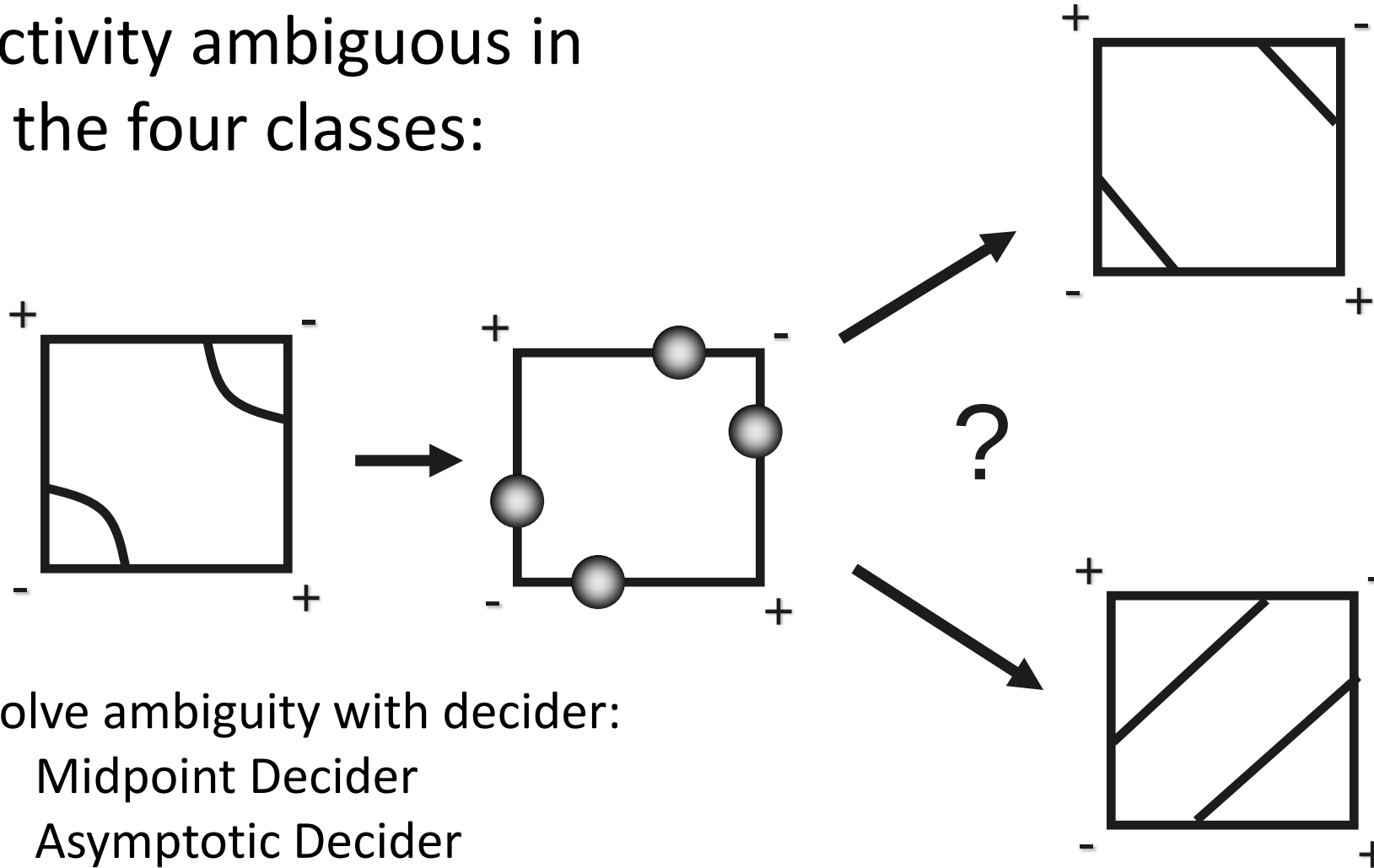
Marching Squares Case Table

- Classify cases according to patterns of signs
 - Symmetries: rotation, polarity $+\leftrightarrow -$
 - Reduces 16 to only 4 unique cases

Case	Polarity	Rotation	Total	
No Crossings	x2		2	
Singlet	x2	x4	8	 (x2 for polarity)
Double adjacent	x2	x2 (4)	4	
Double Opposite	x2	x1 (2)	2	
			$16 = 2^4$	

Marching Squares: Ambiguous Cases

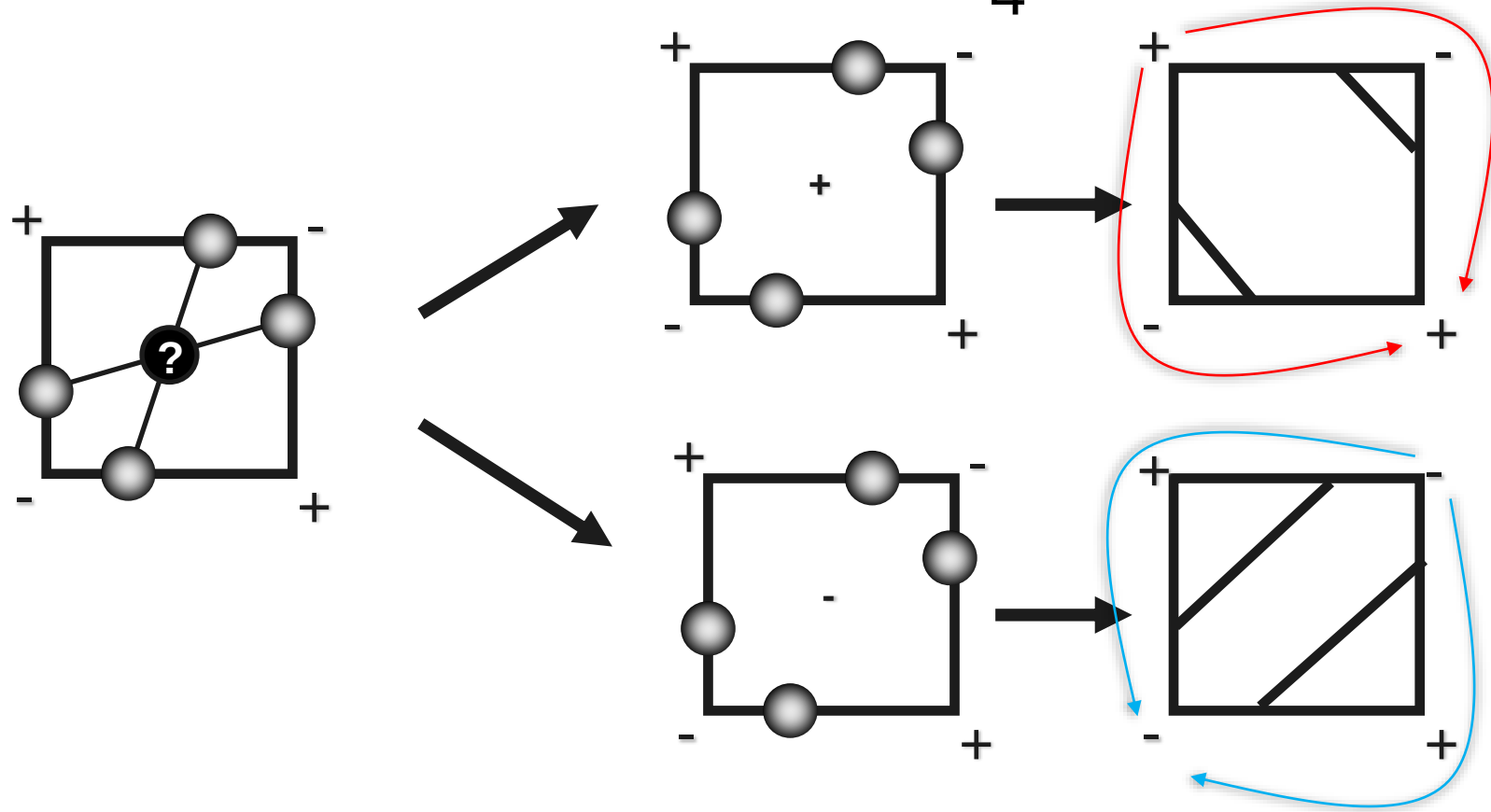
- Connectivity ambiguous in one of the four classes:



Marching Squares: Mid Point Decider

- Mid point decider

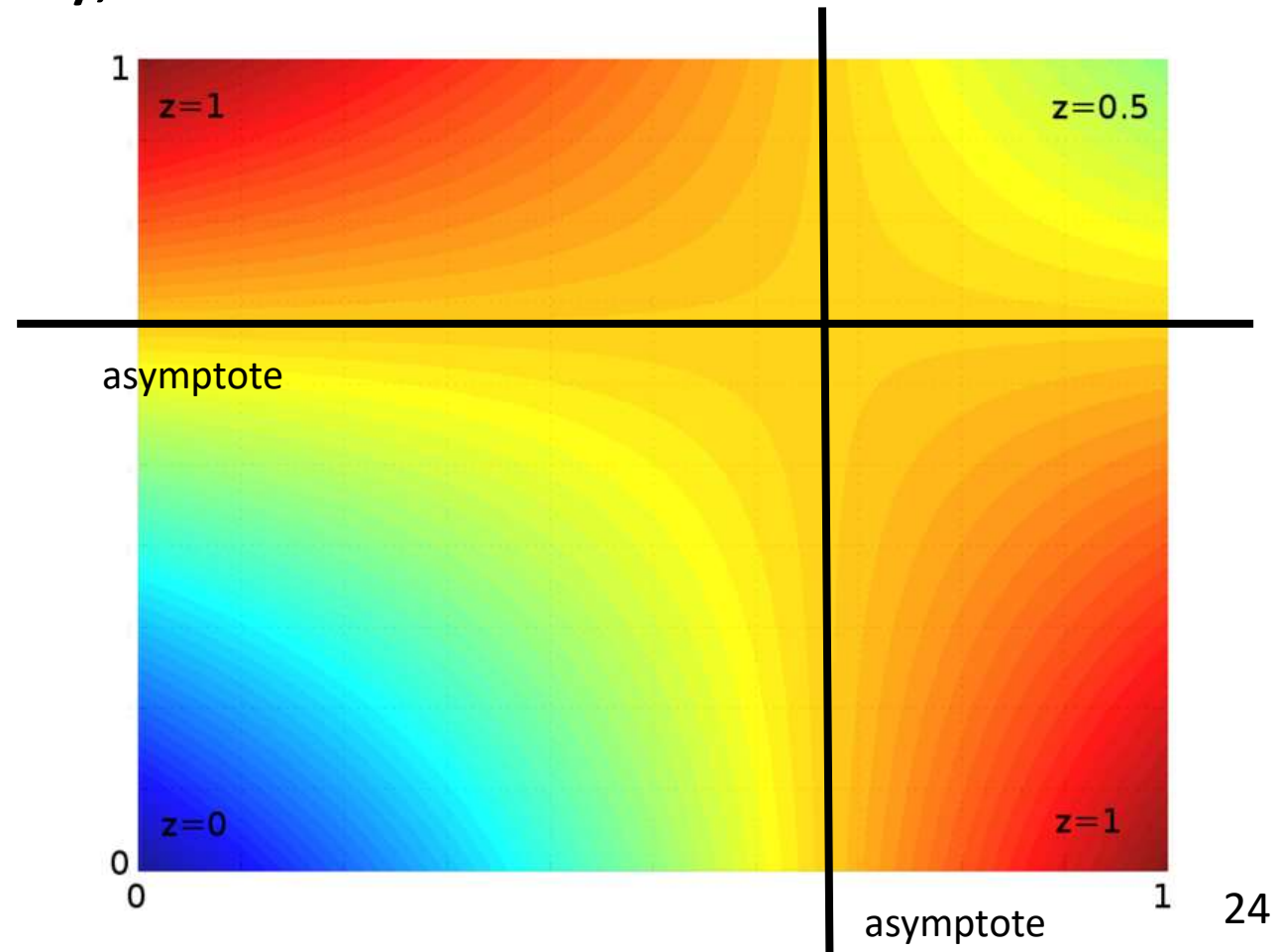
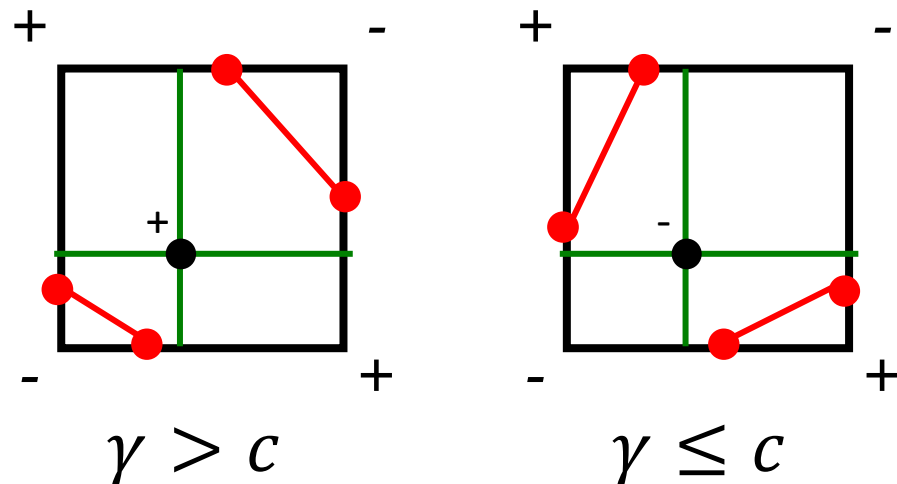
- Bilinear interpolation at the center $f_{\text{center}} = \frac{1}{4}(f_{i,j} + f_{i+1,j} + f_{i,j+1} + f_{i+1,j+1})$



- Does not always yield the same connection as bilinear interpolation

Marching Squares: Asymptotic Decider

- Asymptotic decider
 - When interpolating bilinearly, the true isocurves within a cell are hyperbolas
 - *Idea*: Decide based on the value γ where asymptotes intersect



Asymptotic Decider: Intersection Point

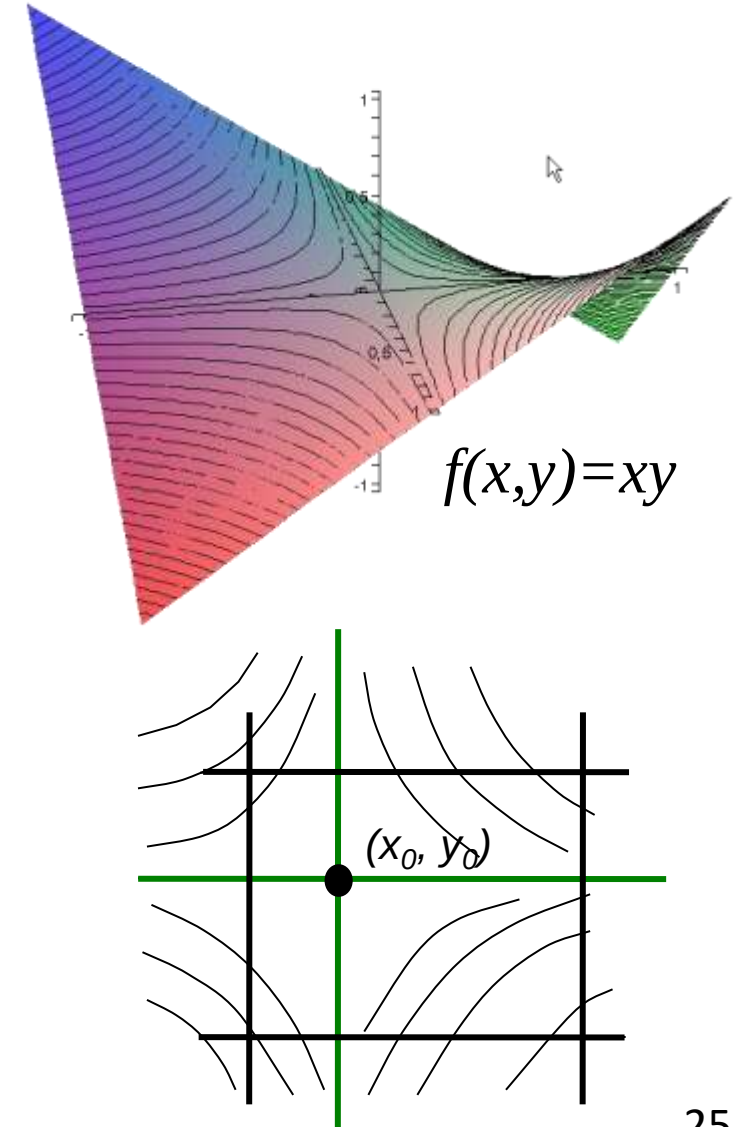
- Interpolate the function bilinearly

$$f(x, y) = f_{i,j}(1-x)(1-y) + f_{i+1,j}x(1-y) + f_{i,j+1}(1-x)y + f_{i+1,j+1}xy$$

- Transform f to

$$f(x, y) = \eta(x - x_0)(y - y_0) + \gamma$$

- γ is the function value at the intersection point of the asymptotes



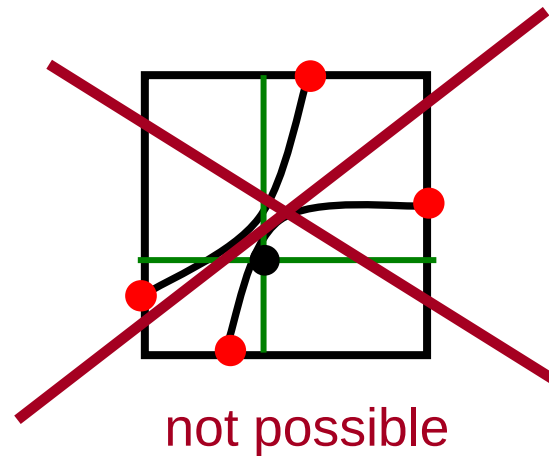
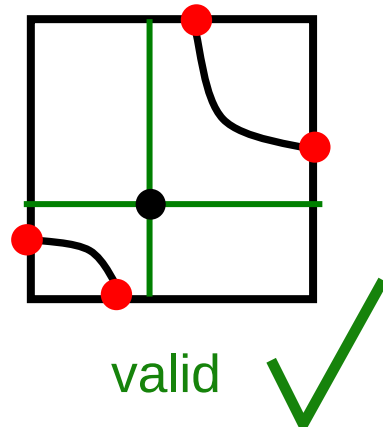
Asymptotic Decider: Implementation

- Explicit transformation of f to

$$f(x, y) = \eta(x - x_0)(y - y_0) + \gamma$$

can be avoided

- *Idea*: investigate the order of intersection points either along x or y axis
- Build pairs of first two and last two intersections



Summary: Isocurves

- In most cases, isocurves of 2D scalar fields are **smooth curves**
 - *Condition:* f is differentiable and $\nabla f \neq 0$
- Isocurves are used as a **fundamental visualization technique**
 - e.g., height lines on topographic maps
- **Marching Squares algorithm** can be used to extract isocurves
 - Find **intersection points** with grid edges based on bilinear interpolation
 - Connect intersection points using a **case table**
 - Resolve ambiguities based on **asymptotic decider**

9.3 Computing Spatial Derivatives

Derivatives in Visualization

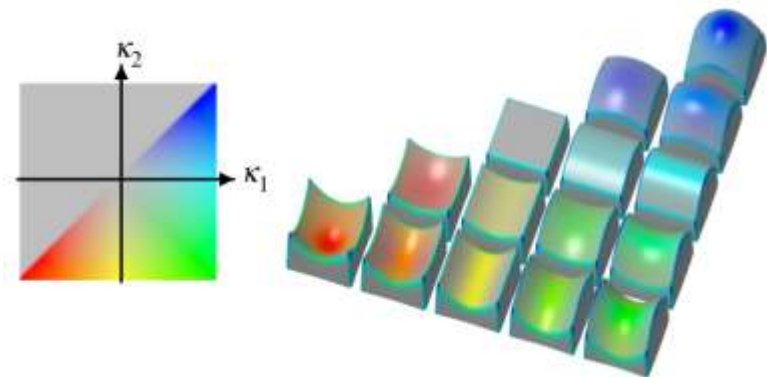
- With suitable interpolation, our data can be represented as a smooth function $f(\mathbf{x})$
- Taking derivatives of $f(\mathbf{x})$ provides us with information about changes in a small neighborhood of $\mathbf{x}$
- *Example:* Scalar Field



Isosurface: Only Values



Gradient: Shading



Hessian: Curvature

Derivative of a Function

- **Reminder:** The **derivative** of a function is defined from the difference quotient as

$$\frac{d}{dt}f(t) := \lim_{h \rightarrow 0} \frac{f(t+h) - f(t)}{h}$$

- If this limit exists, the function is called **differentiable** (on the set on which the limit converges). Other notation:

$$\frac{d}{dt}f(t) = f'(t) = \dot{f}(t)$$

↑
if reference is
obvious from context

↖
if t denotes time

Repeated Differentiation

- Higher-Order Differentiation:

- The differentiation operation can be applied repeatedly:

$$\frac{d^2}{dt^2} f(t) = f''(t) = \ddot{f}(t) = \lim_{h \rightarrow 0} \frac{f(t+h) - 2f(t) + f(t-h)}{h^2}$$

$$\frac{d^k}{dt^k} f(t) = f^{(k)}(t)$$

- A function whose k -th derivative exists and is continuous belongs to the set C^k .

Slight abuse of notation:

- C^0 means just continuity.
 - C^{-1} means “not even continuous”.
 - C^∞ means “differentiable any number of times”.

Partial Derivative

- If a function depends on more than one variable:
 - The notation changes, the rest remains the same:

$$\frac{\overset{\text{use curly-d}}{\partial}}{\partial x_k} f(x_1, \dots, x_{k-1}, x_k, x_{k+1}, \dots, x_n) := \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{k-1}, x_k + h, x_{k+1}, \dots, x_n) - f(x_1, \dots, x_{k-1}, x_k, x_{k+1}, \dots, x_n)}{h}$$

- Derivative in one variable x_k only, other variables remain constant.
- This is called a **partial derivative**. Other notation:

$$\frac{\partial}{\partial x_k} f(\mathbf{x}) = \partial_k f(\mathbf{x}) = f_{x_k}(\mathbf{x})$$

Gradient

- Gradient:
 - Given a function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ (scalar field)
 - The vector of all partial derivatives of f is called the **gradient**
 - It indicates the direction in which values of f change most rapidly
 - Its magnitude indicates the speed at which values change along that direction

$$\nabla f(\mathbf{x}) = \begin{pmatrix} \frac{\partial}{\partial x_1} f(\mathbf{x}) \\ \vdots \\ \frac{\partial}{\partial x_n} f(\mathbf{x}) \end{pmatrix}$$

Hessian Matrix

- Second order partial derivatives
 - Have to consider all combinations of partial derivatives:

$$f(\mathbf{x}) \equiv H_f(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2}{\partial x_1^2} & \frac{\partial}{\partial x_1} \frac{\partial}{\partial x_2} & \cdots & \frac{\partial}{\partial x_1} \frac{\partial}{\partial x_n} \\ \frac{\partial}{\partial x_2} \frac{\partial}{\partial x_1} & \frac{\partial^2}{\partial x_2^2} & \cdots & \frac{\partial}{\partial x_2} \frac{\partial}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_n} \frac{\partial}{\partial x_1} & \frac{\partial}{\partial x_n} \frac{\partial}{\partial x_2} & \cdots & \frac{\partial^2}{\partial x_n^2} \end{pmatrix}$$

- “Hessian” matrix (symmetric for $f \in C^2$)

Directional / Higher-Order Derivatives

- **Directional Derivatives:**

- For a smooth function f , we can obtain derivatives in direction $\hat{\mathbf{v}}$ via gradient and Hessian:

$$\frac{\partial f}{\partial \hat{\mathbf{v}}} = \nabla f \cdot \hat{\mathbf{v}} \quad \frac{\partial^2 f}{\partial \hat{\mathbf{v}}^2} = \hat{\mathbf{v}}^T \mathbf{H}_f \hat{\mathbf{v}}$$

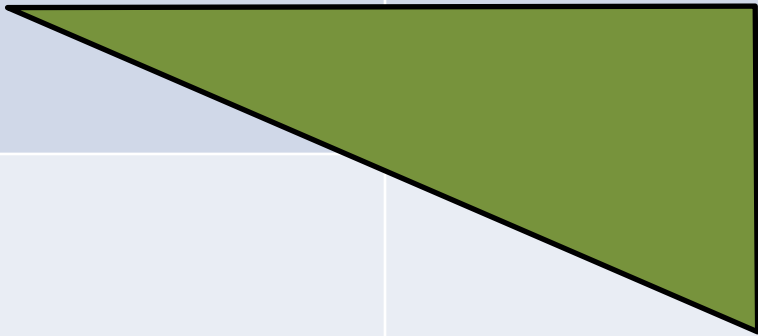
- **Higher-Order Derivatives:**

- Can do all combinations: $\frac{\partial}{\partial x_{i_1}} \frac{\partial}{\partial x_{i_2}} \cdots \frac{\partial}{\partial x_{i_k}} f$
- Produces a higher-order tensor (k -D array)
- Order does not matter for $f \in C^k$

Data Type vs. Attribute

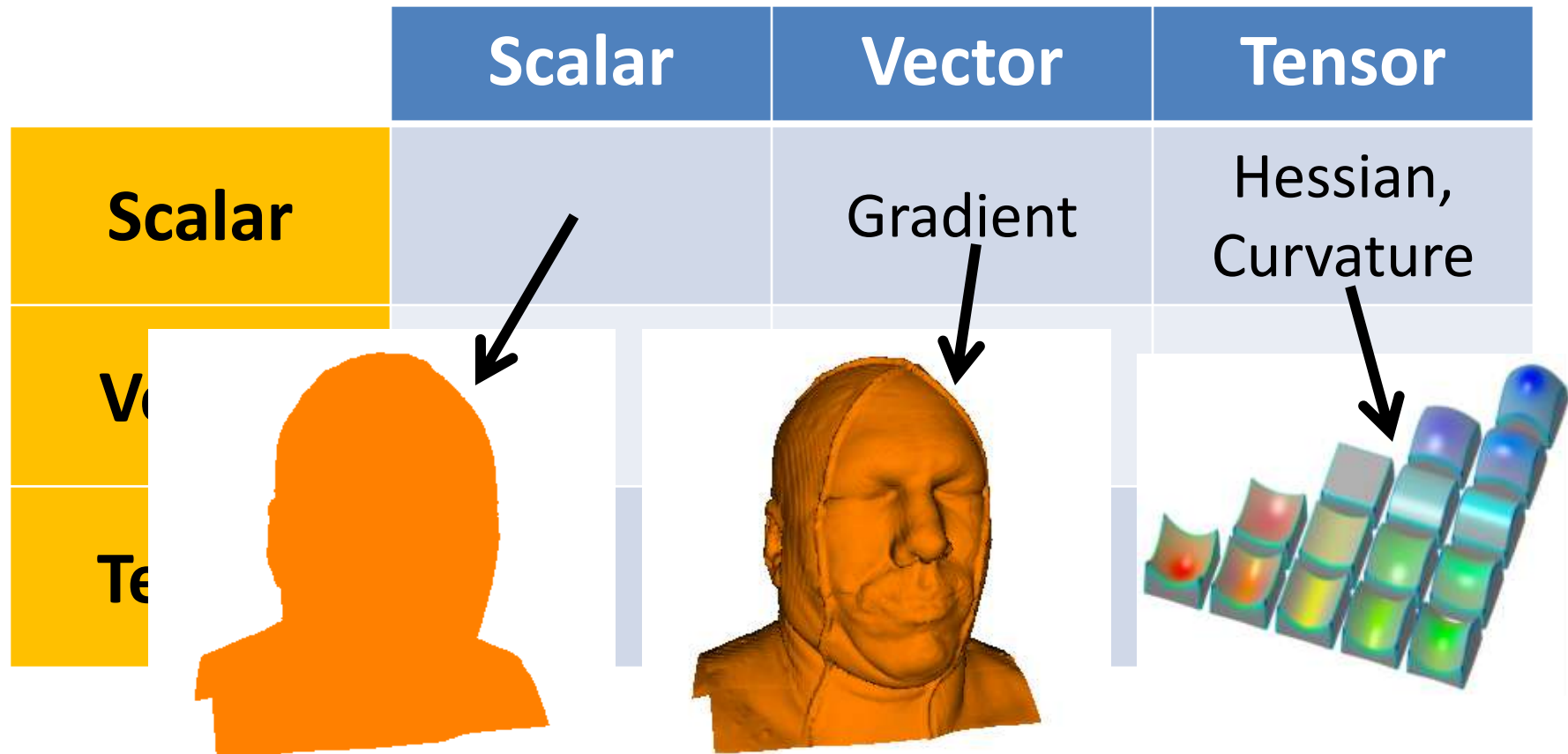
Differentiation provides **data attributes** that have higher order than the underlying **data type**:

	Scalar	Vector	Tensor
Scalar			
Vector			
Tensor			



Data Type vs. Attribute

Differentiation provides **data attributes** that have higher order than the underlying **data type**:



Differentiation on Discrete Data

- Remember that the derivative $f'(x)$ is defined as the limit of the difference quotient:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

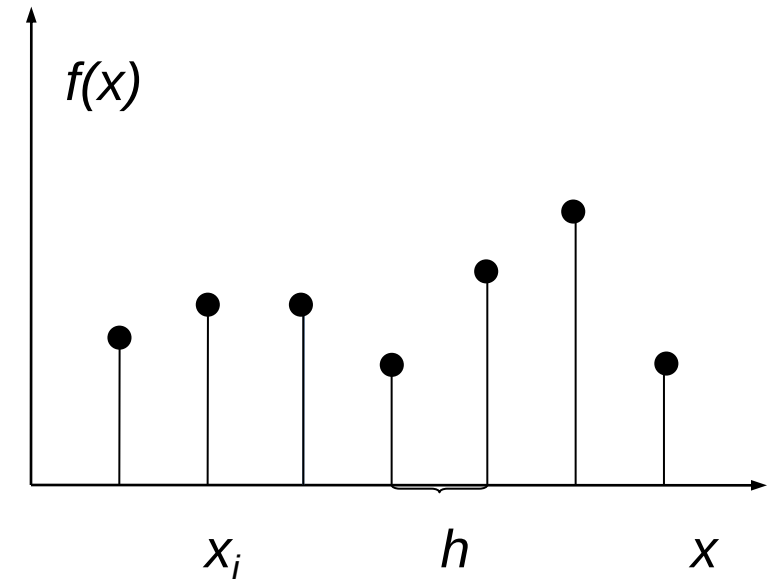
- If $f(x)$ is sampled at (sorted) positions x_i , it is natural to approximate $f'(x)$ using finite differences, which can be a
 - Forward difference:** $f'_f(x_i) = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$
 - Backward difference:** $f'_b(x_i) = \frac{f(x_i) - f(x_{i-1})}{x_i - x_{i-1}}$

Central Differences

- Another alternative is the **central difference**:

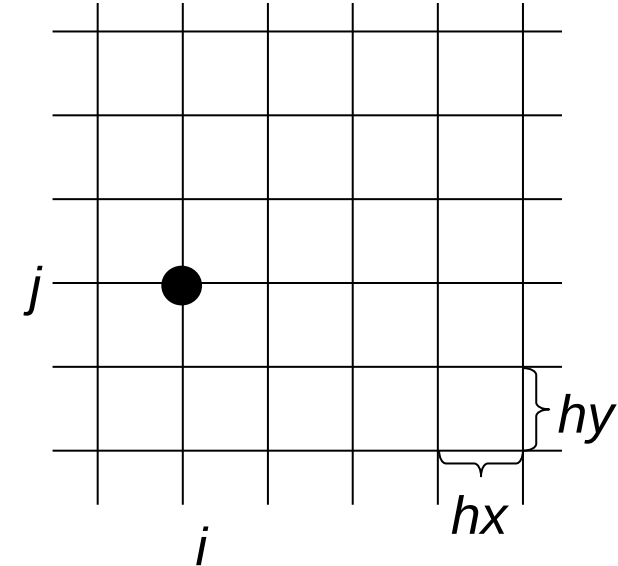
$$f'_c(x_i) = \frac{f(x_{i+1}) - f(x_{i-1}))}{x_{i+1} - x_{i-1}}$$

- When data is given on a **uniform grid**, the denominator simplifies to h (intermediate = forward/backward) or $2h$ (central differences)
- **Approximation error** in intermediate differences decreases linearly with decreasing h .
With central differences, it decreases quadratically.
 - Exercise to show why!



Discrete Partial Derivatives

- Same finite difference schemes as in univariate case, applied separately along each coordinate axis



- *Example:* gradient in a uniform grid via central differences

$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \end{pmatrix} = \begin{pmatrix} \frac{f_{i+1,j,k} - f_{i-1,j,k}}{2hx} \\ \frac{f_{i,j+1,k} - f_{i,j-1,k}}{2hy} \\ \frac{f_{i,j,k+1} - f_{i,j,k-1}}{2hz} \end{pmatrix}$$

Stencil Notation

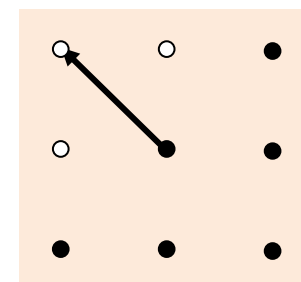
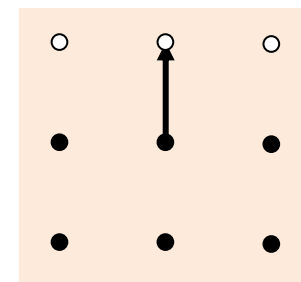
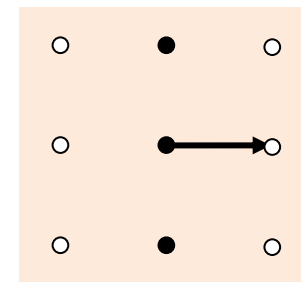
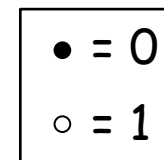
- Finite difference equations often written as “**stencil**” (cross-correlation kernel)
- 2D examples for central differences:

$$\frac{\partial}{\partial x} = \frac{1}{2h} \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \frac{\partial}{\partial y} = \frac{1}{2h} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

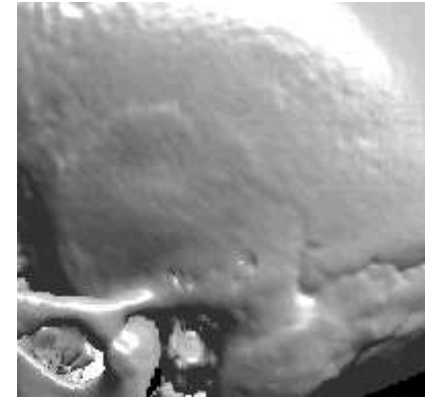
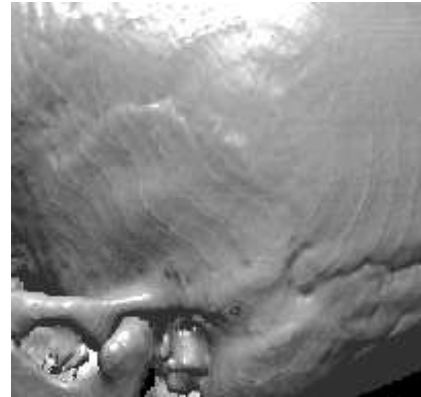
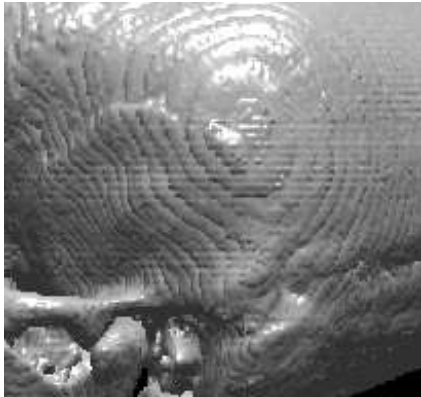
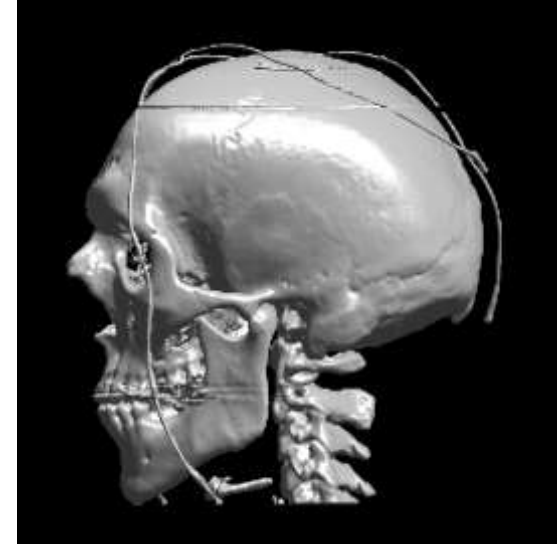
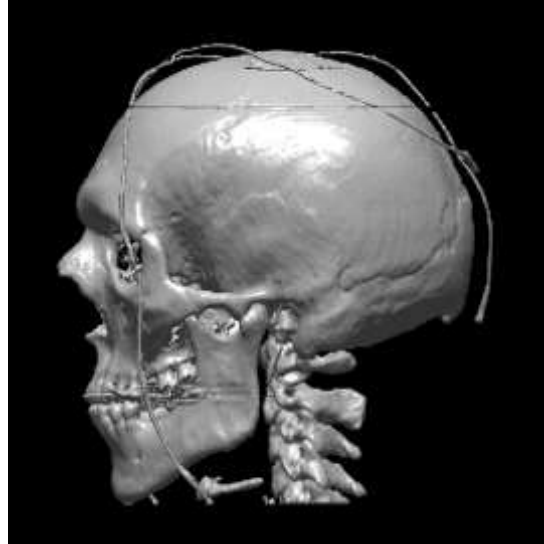
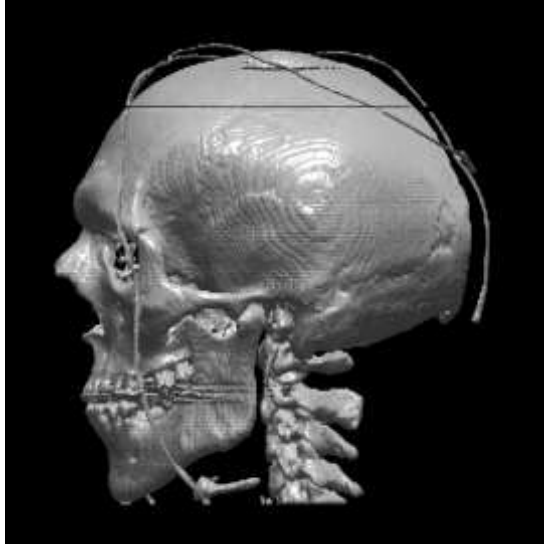
$$\frac{\partial^2}{\partial x^2} = \frac{1}{h^2} \begin{bmatrix} 0 & 0 & 0 \\ 1 & -2 & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \frac{\partial^2}{\partial x \partial y} = \frac{1}{4h^2} \begin{bmatrix} -1 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & -1 \end{bmatrix}$$
$$\frac{\partial^2}{\partial y^2} = \frac{1}{h^2} \begin{bmatrix} 0 & 1 & 0 \\ 0 & -2 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Intermediate vs. Central Differences

- Intermediate differences are able to detect **higher frequencies**
 - This also increases the effects of noise
- Central differences are **more expensive** to compute than intermediate differences
 - need 6 rather than 3 additional numbers to compute 3D gradient
- Neither of the two are **isotropic**:
 - Depending on edge orientation, the gradient norm at an edge in a 3D binary image varies between 0.5 and $\frac{\sqrt{3}}{2}$ (assuming central differences with $dx=dy=dz=1$)



Visual Comparison of Gradient Schemes



Forward/Backward
Differences

Central Differences

Sobel Operator

Sobel Operator

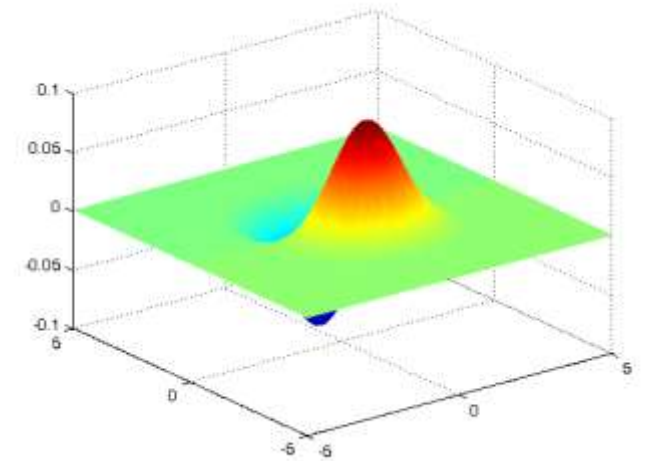
- The **Sobel Operator** is based on averaging central differences over a spatial neighborhood
 - Smoother, more isotropic (but still not perfectly)
 - 3D gradient requires all values in a 3x3x3 neighborhood
 - But: less expensive than most convolution-based approaches (next)
 - Stencil for $\frac{\partial}{\partial x}$ in 3D data:

	prev. slice (z_0-1)	this slice (z_0)	next slice (z_0+1)
$\frac{1}{32h} \times$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -2 & 0 & 2 \\ -4 & 0 & 4 \\ -2 & 0 & 2 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$

Differentiation by Convolution

- If we use convolution to obtain a smooth interpolation $f(\mathbf{x})$, it's natural to differentiate it
 - Amounts to **convolving with derivative of kernel**:
$$\frac{d(g * h)(t)}{dt} = \frac{d \int_{-\infty}^{\infty} g(\tau) h(t - \tau) d\tau}{dt} = \int_{-\infty}^{\infty} g(\tau) \frac{dh(t - \tau)}{dt} d\tau$$
$$= (g * h')(t)$$
 - When using **separable convolution** in higher dimension, partial differentiation only affects one of the kernels:

$$\frac{d(g * h)(\mathbf{t})}{dt_1} = \iint g(\tau_1, \tau_2) h'_1(t_1 - \tau_1) h_2(t_2 - \tau_2) d\boldsymbol{\tau}$$



Interpolating Gradient Magnitude

- Computing gradient magnitude $\|\nabla f\|$ is a non-linear operation and thus does not commute with interpolation
 - First interpolate gradient, then take magnitude!
 - *Reason:* Squaring doubles spatial frequencies

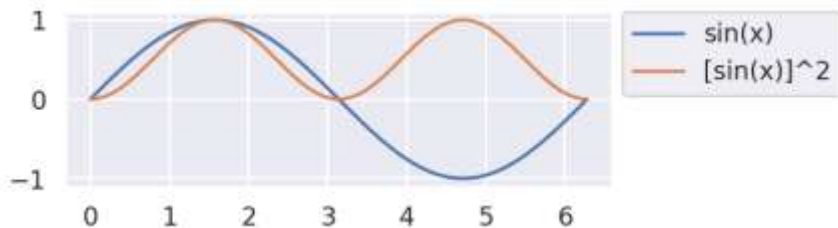
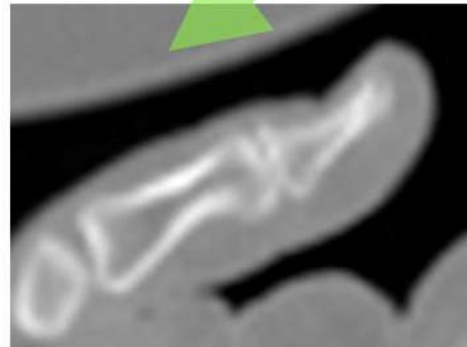


image = $f[\mathbf{x}]$

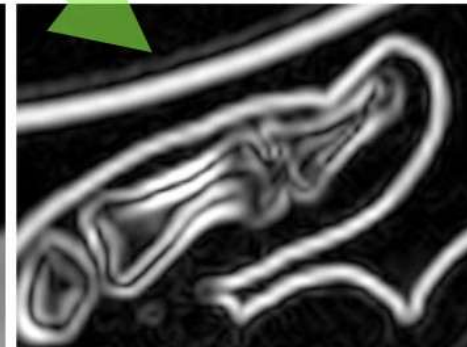


$g[\mathbf{x}] = \|\nabla f[\mathbf{x}]\|$

a non-linear function of $f[\mathbf{x}]$



$f[\mathbf{x}] * k(\mathbf{x})$



$\|\nabla(f[\mathbf{x}] * k(\mathbf{x}))\|$



$g[\mathbf{x}] * k(\mathbf{x})$

Differentiation: What You Should Know

- Brief recap of **multivariate calculus** (gradients, Hessian)
- **Finite difference schemes** are widely used to compute derivatives on uniform grids
 - Stencil notation is a concise way of writing them
- **Convolution-based interpolation** implies a natural way of taking derivatives
- **Sobel operator** provides a computationally cheaper but still quite smooth alternative

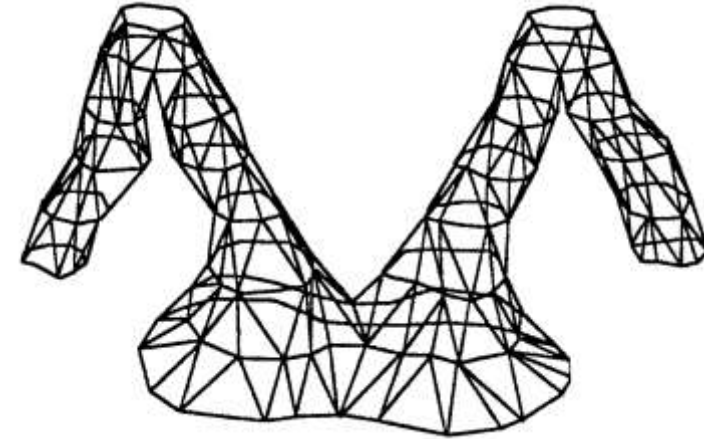
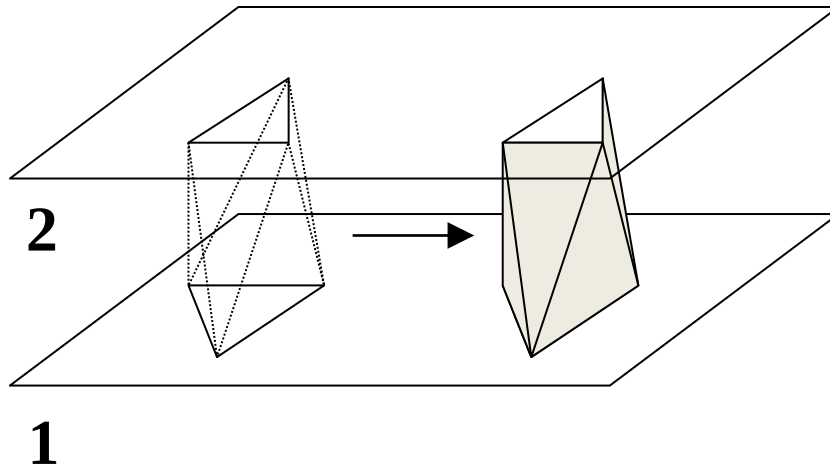
Section 9.4: Isosurface Extraction

Techniques for Isosurface Extraction

- **Goal:** Obtain an explicit representation of the isosurface $f(x,y,z) = c$ with isovalue c
- **Techniques** to determine and to reconstruct isosurfaces from volume data
 - Contour tracing
 - Marching cubes
 - Marching tetrahedra

Contour Tracing

- Build isosurfaces from 2D contours:
 - *Find isocurves* in 2D slices and represent them as polylines
 - *Connect contours* from adjacent slices and form triangles



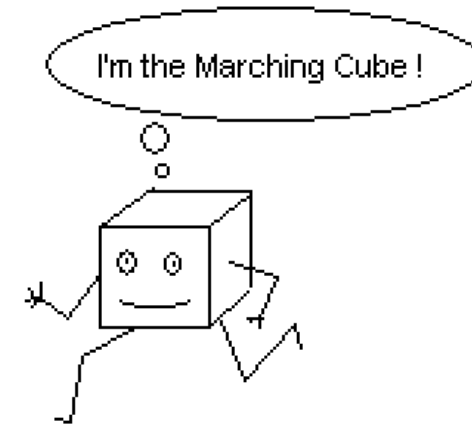
- Problems:
 - Sometimes there are many contours in each slice or there is a high variation between slices
 - Tracing (assignment) becomes very difficult

Marching Cubes

- In order to get a better approximation of the „real“ isosurface, the **Marching Cubes** (MC) algorithm was developed [Lorensen, Cline 1987]
 - Works on the original data (voxel grid)
 - Approximates the surface by a triangle mesh
 - Surface is found by linear interpolation along cell edges
 - Uses gradients as the normals of the isosurface
 - Efficient computation by means of lookup tables
- THE standard isosurface extraction algorithm
 - Cited more than 21,000 times as of May 2025 (google scholar)

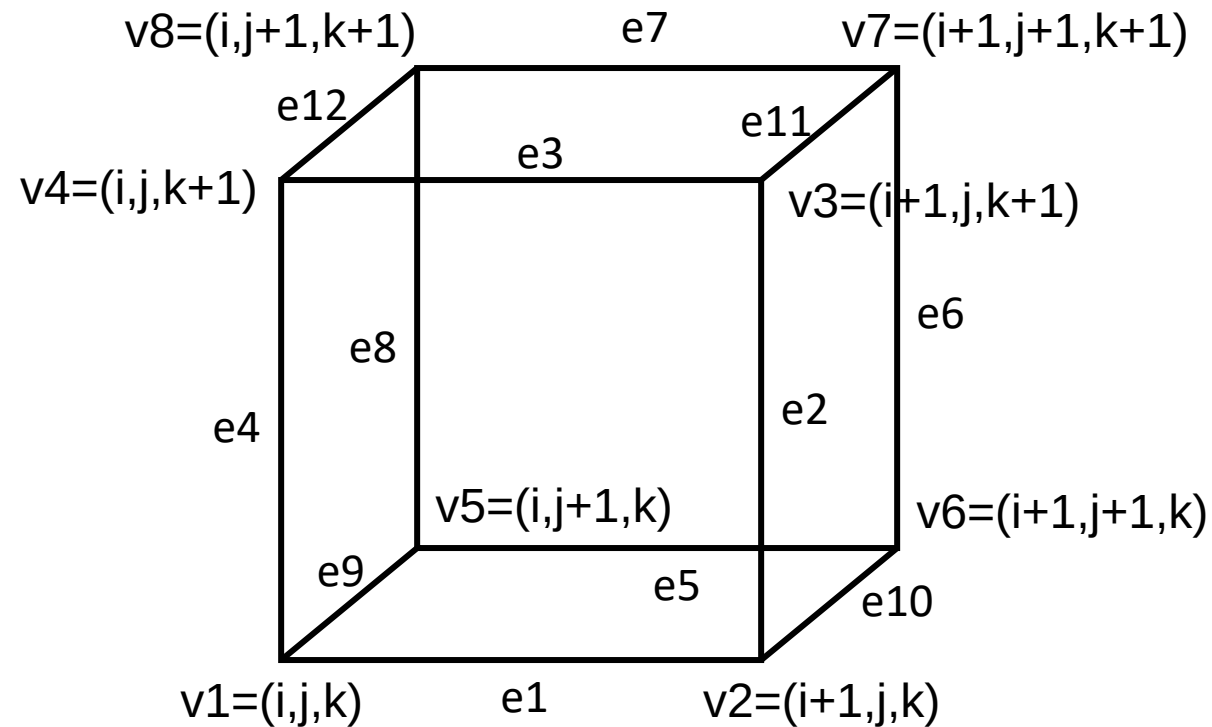
Marching Cubes: Core Algorithm

- The core MC algorithm:
 1. Consider a cell
 2. Classify each vertex as inside or outside
 3. Build an index
 4. Get edge list from table[index]
 5. Interpolate the edge location
 6. Compute gradients
 7. Consider ambiguous cases
 8. Go to next cell



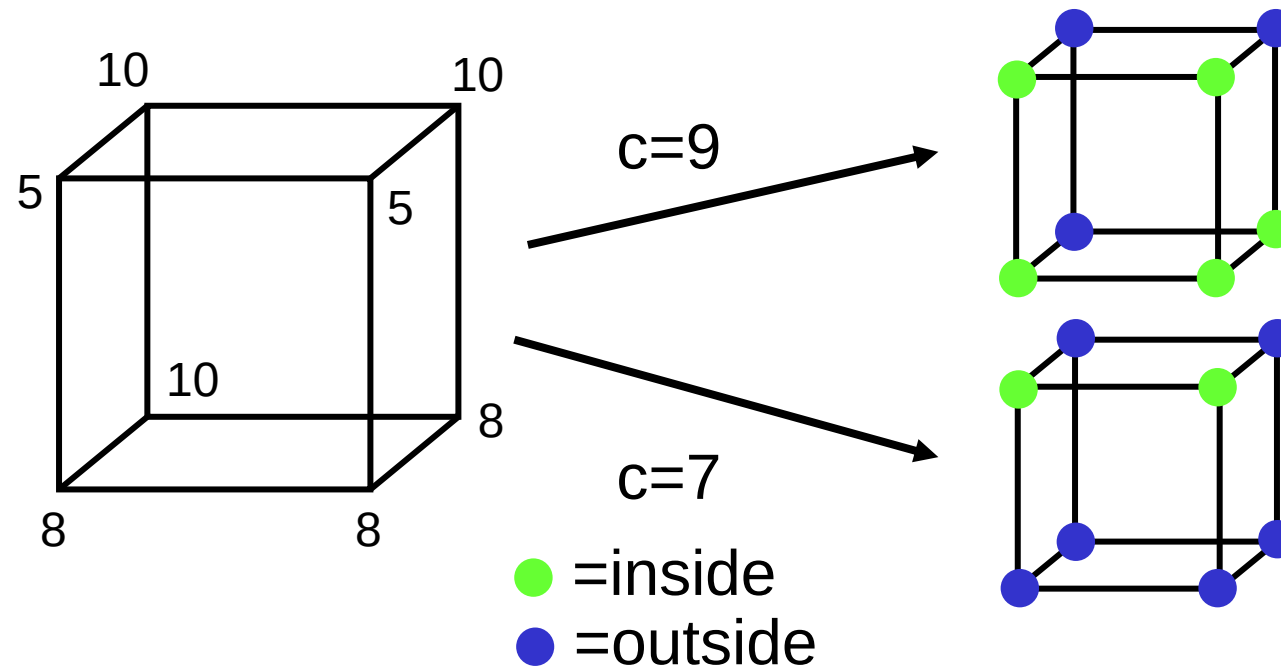
Marching Cubes: Step 1

- Consider a cell defined by eight data values



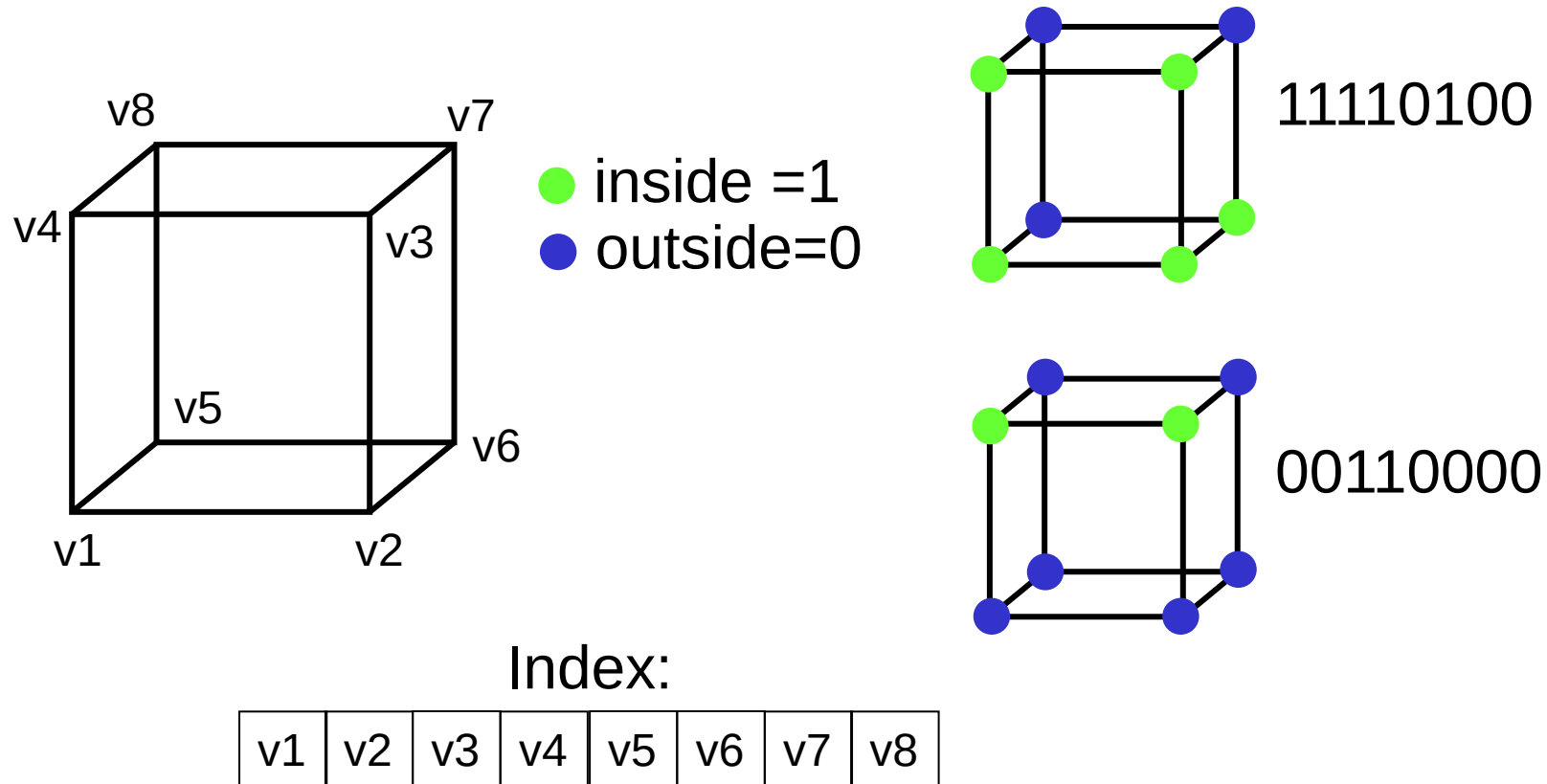
Marching Cubes: Step 2

- Classify each voxel according to whether it lies
 - outside the surface (value > isosurface value)
 - inside the surface (value $\leq$ isosurface value)



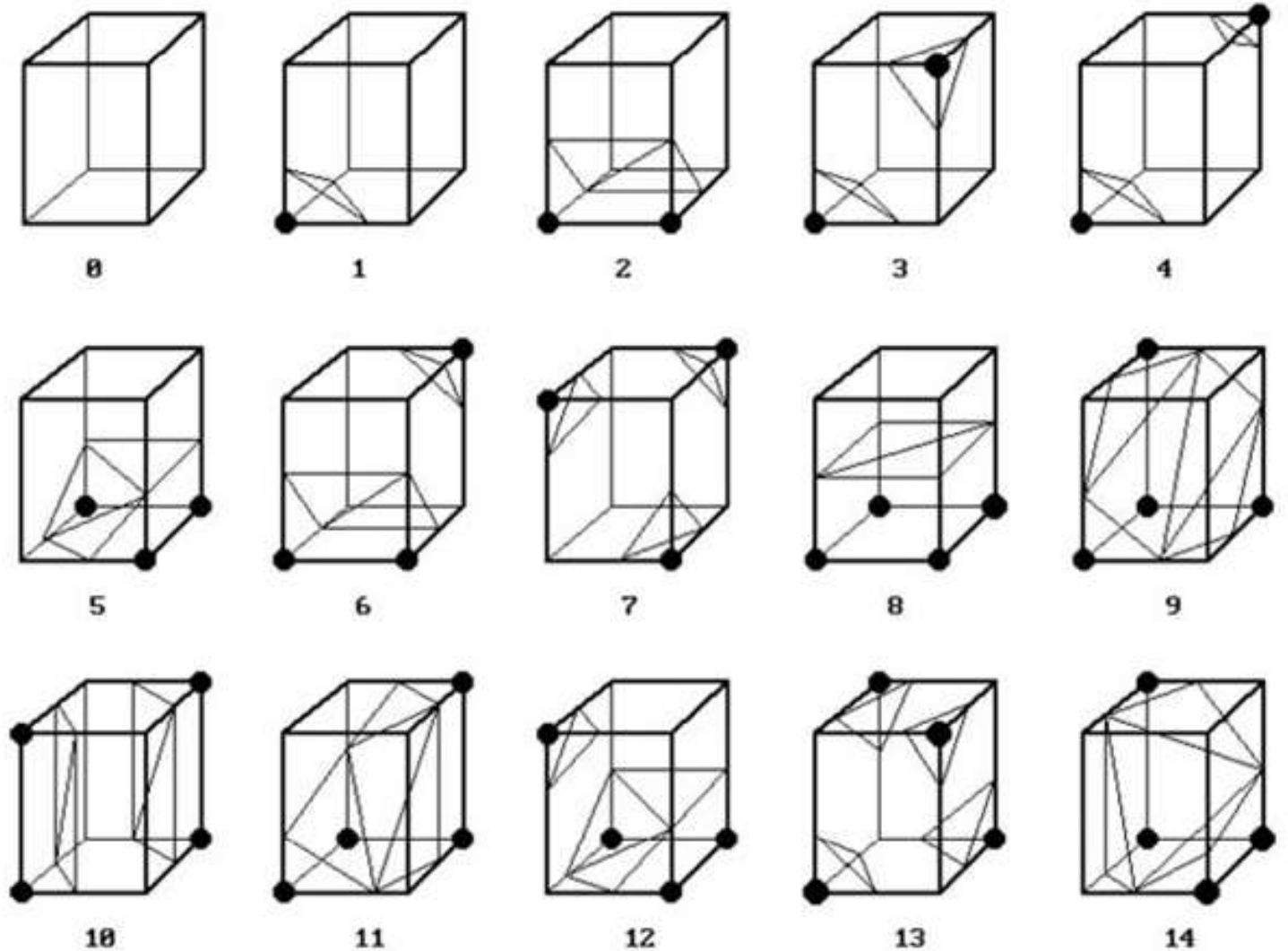
Marching Cubes: Step 3

- Use the binary labeling of each voxel to create an index



Marching Cubes: Step 4

- For a given index, access an array storing a list of edges
 - All 256 cases can be derived from 15 base cases due to symmetries



Marching Cubes: Step 4 cont.

- Get edge list from table
 - Example for

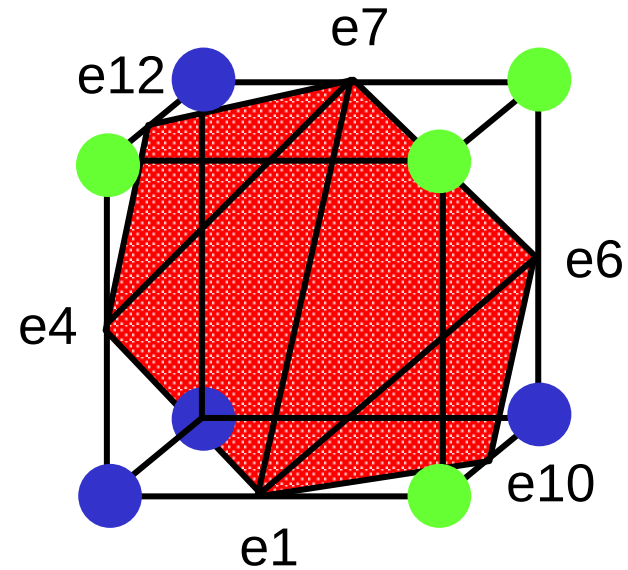
Index = 01110010

triangle 1 = e4,e7,e12

triangle 2 = e1, e7, e4

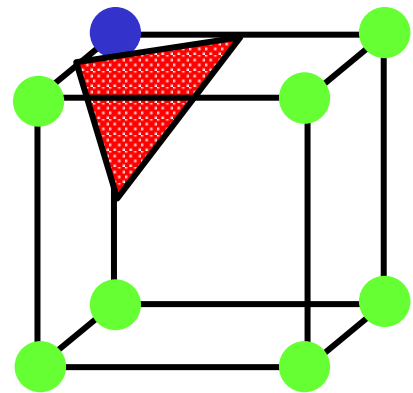
triangle 3 = e1, e6, e7

triangle 4 = e1, e10, e6



Marching Cubes: Step 5

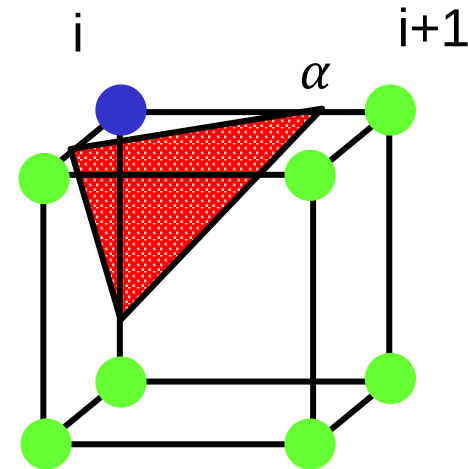
- For each triangle edge, find the vertex location along the edge using linear interpolation of the voxel values



● = 10
● = 0

$c=5$

$$\alpha = i + \left(\frac{c - v[i]}{v[i+1] - v[i]} \right)$$

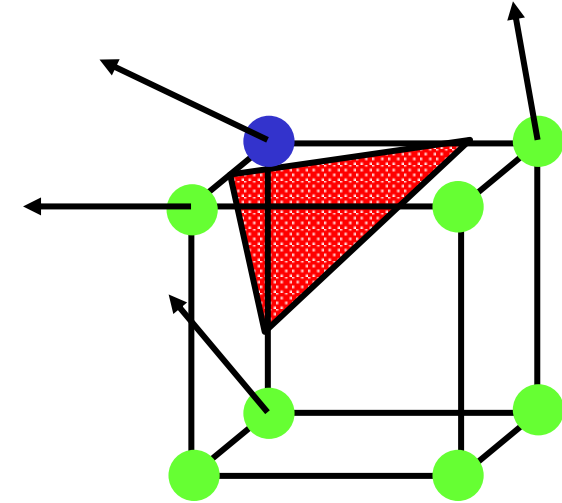


$c=8$

Marching Cubes: Step 6

- Calculate the normal **n** at each cube vertex
 - E.g., using central differences:

$$-\nabla f = \begin{pmatrix} \frac{f_{x+1,y,z} - f_{x-1,y,z}}{2\Delta x} \\ \frac{f_{x,y+1,z} - f_{x,y-1,z}}{2\Delta y} \\ \frac{f_{x,y,z+1} - f_{x,y,z-1}}{2\Delta z} \end{pmatrix} \quad \mathbf{n} = \frac{\nabla f}{\|\nabla f\|}$$



- Use linear interpolation to compute the polygon vertex normal (of the isosurface)

Marching Cubes: Step 7

- Consider ambiguous cases
 - Adjacent vertices: different states
 - Diagonal vertices: same state
 - Resolution via asymptotic decider

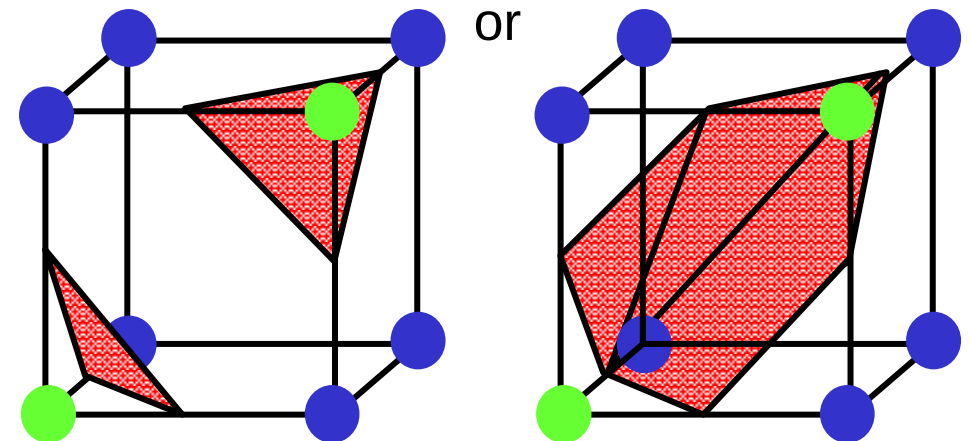
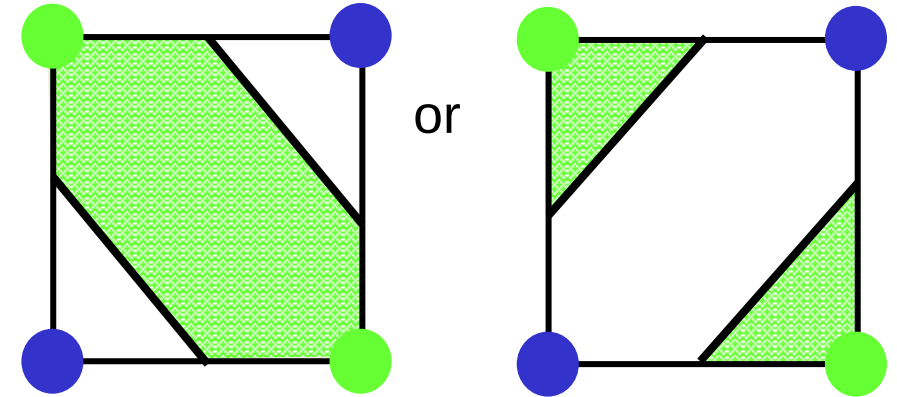


Illustration: Ambiguous Cases

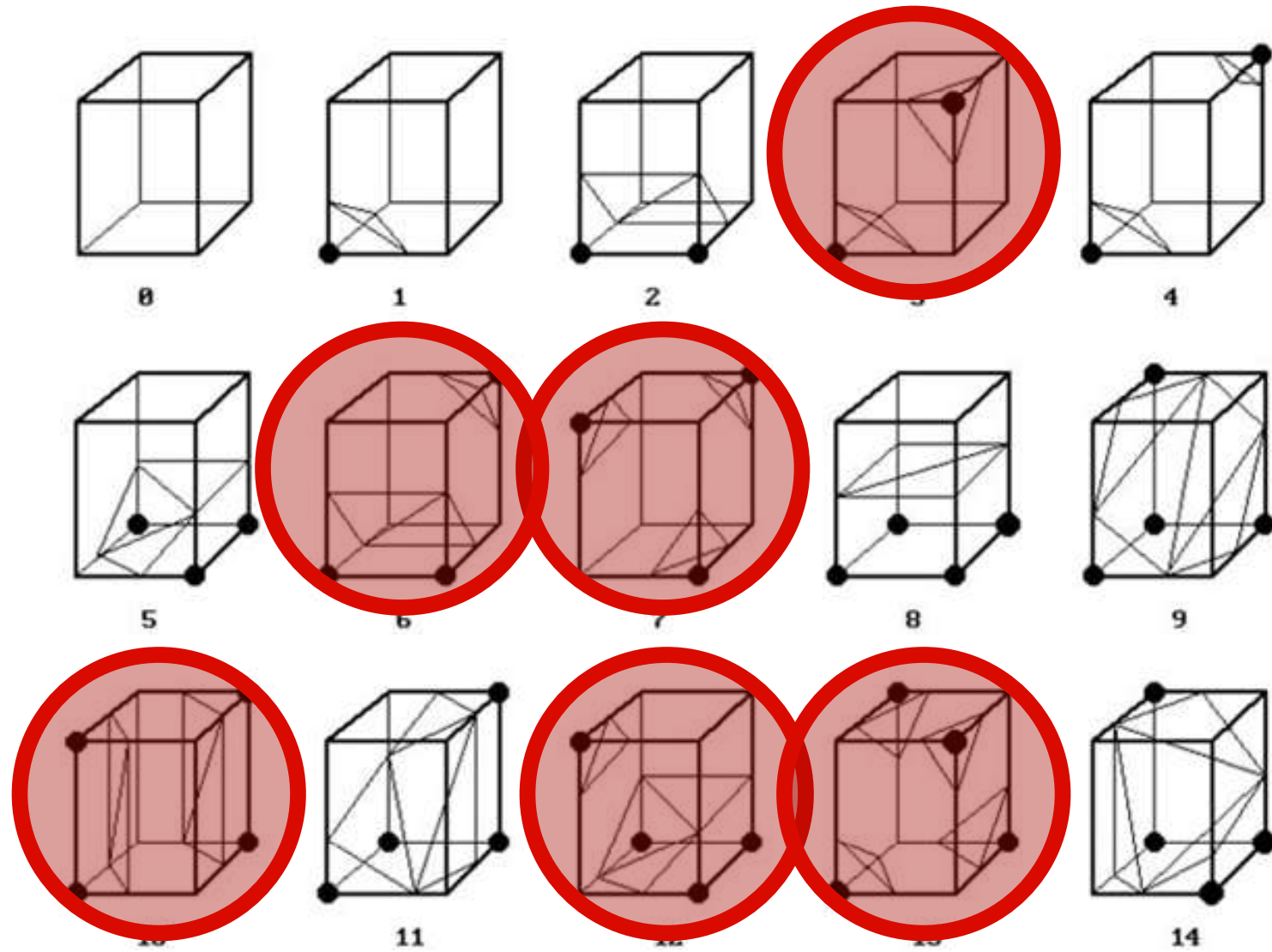
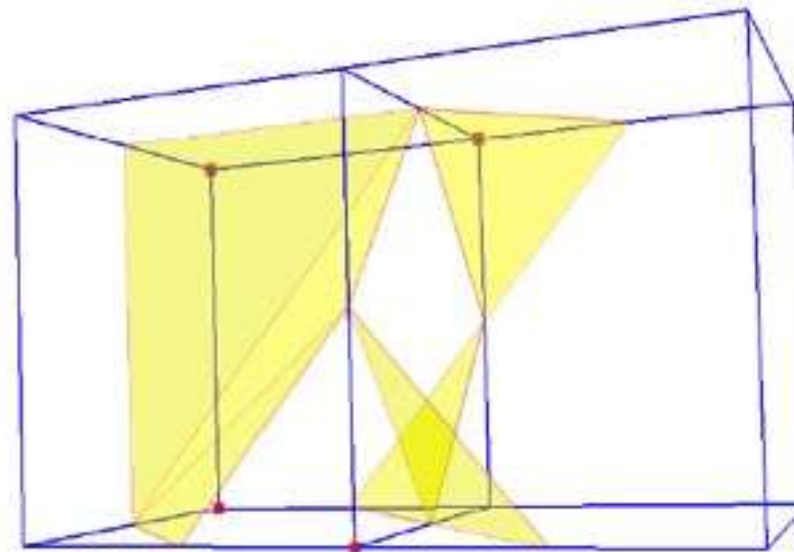


Illustration: Holes From Ambiguous Cases

- Ignoring the problem of ambiguous cases can lead to holes in the resulting surface
- *Example:* Yellow isosurface mesh does not separate red (outside) from blue (inside) vertices



Marching Cubes: Step 7 cont.

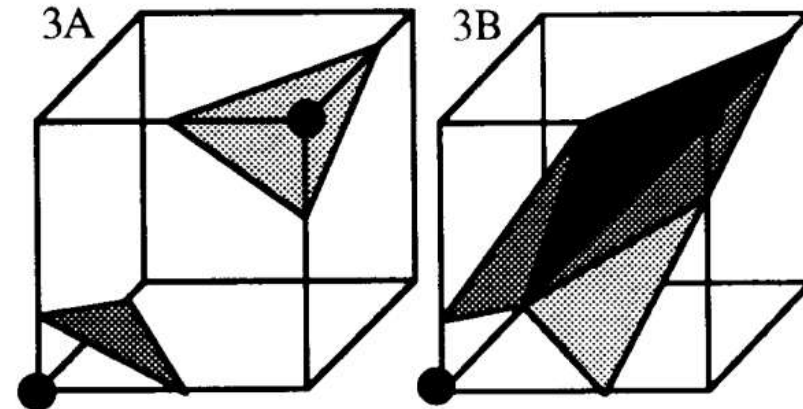
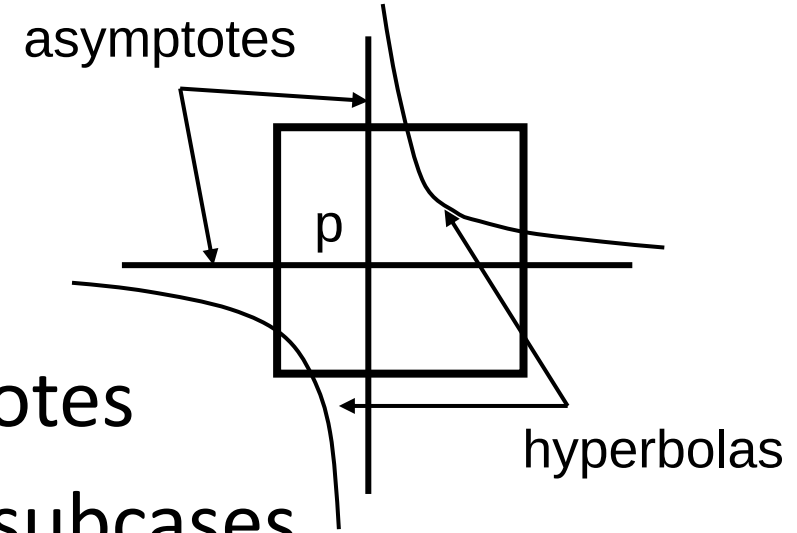
- Asymptotic Decider [Nielson, Hamann 1991]

- Similar to Marching Squares, assumes bilinear interpolation within faces and considers corresponding hyperbolas

- Compute sign at intersection **p** of asymptotes

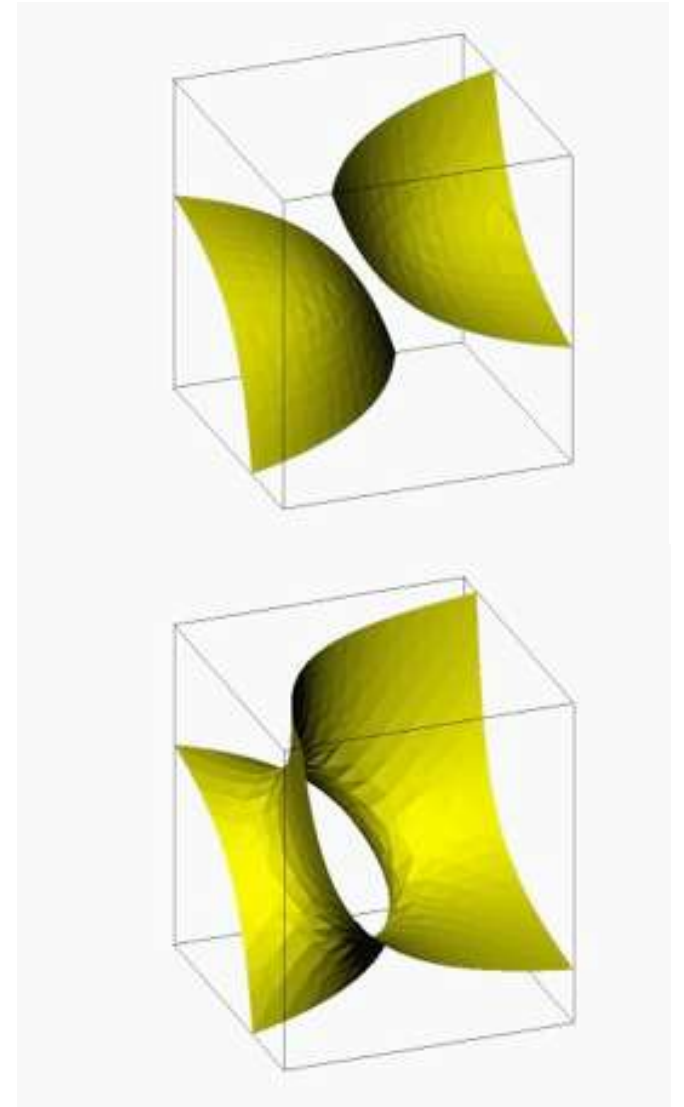
- $\text{Sign}(f(\mathbf{p}) - c)$ splits 6 ambiguous cases into subcases

- Some cases have multiple ambiguous faces
- 18 sub-cases overall, increasing 15 to 27 cases



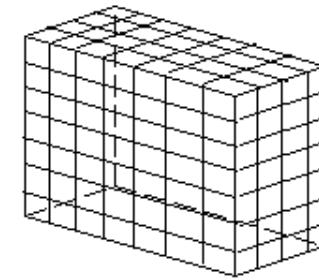
Interior Ambiguities

- Further ambiguities arise in the **interior of cells**
 - *Example:* Opposite corners positive
 - Do not lead to holes in the mesh
 - In order to ensure the same topology as trilinear interpolation, need to further subdivide 6 of the 27 cases
 - “Marching Cubes 33” [Chernyaev 1995]

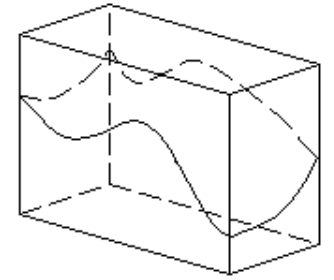


Summary: Marching Cubes Case Table

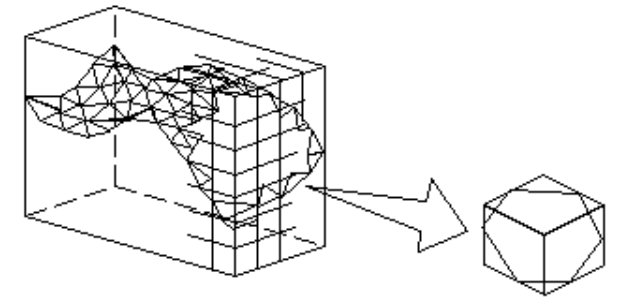
- Marching Cubes case table
 - 256 Cases
 - Reduce to 15 cases by symmetry
 - Ambiguity resides in 6 cases
 - Split into 18 sub-cases
 - Making choices arbitrarily leads to holes in the mesh
 - Up to 5 triangles per cube
- Dataset of 512^3 voxels can result in several million triangles
- Optimizations:
 - Prevent vertex replication
 - Mesh simplification



(a) Volume data



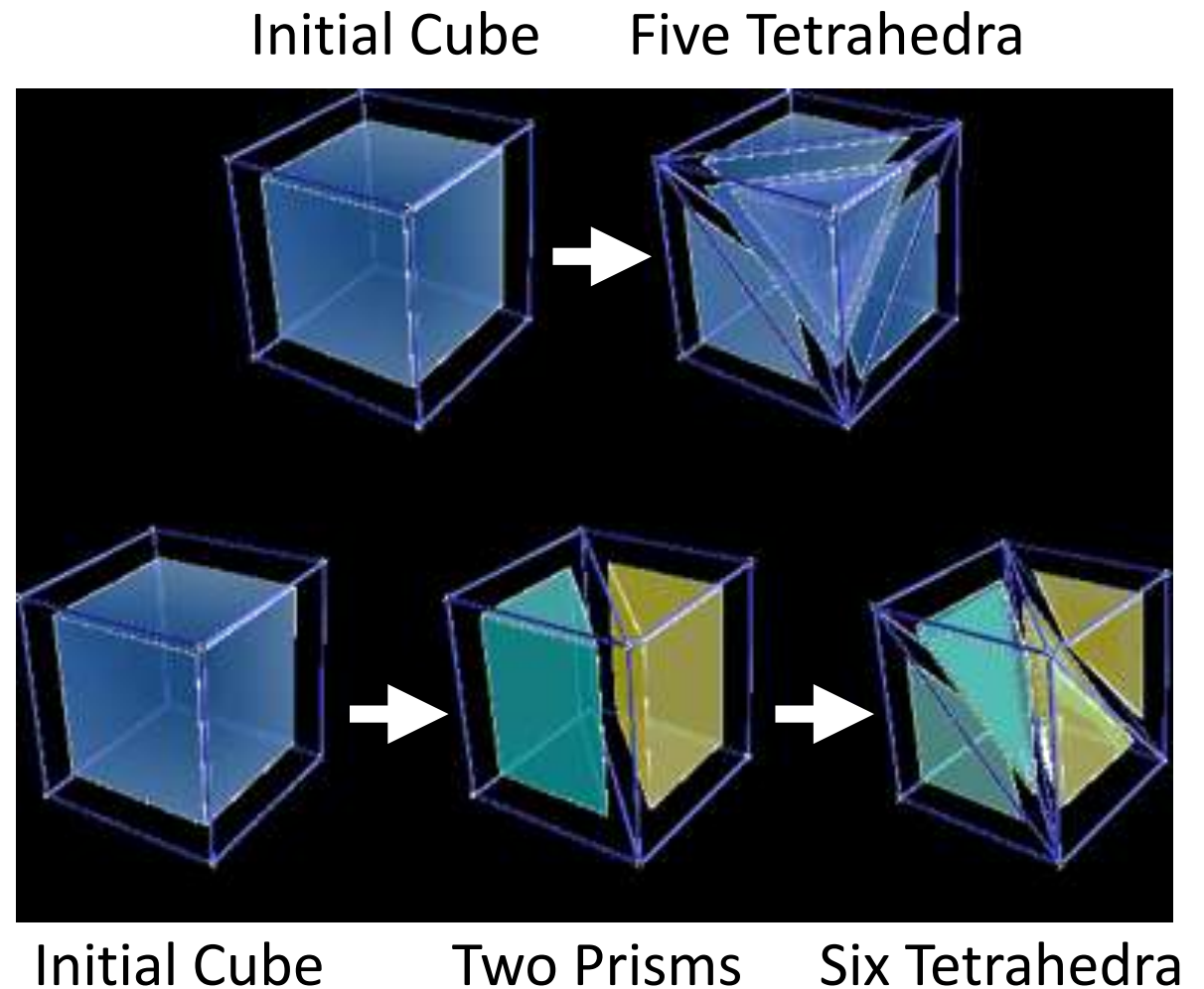
(b) Isosurface
 $S = f(x, y, z)$



(c) Polygonal Approximation

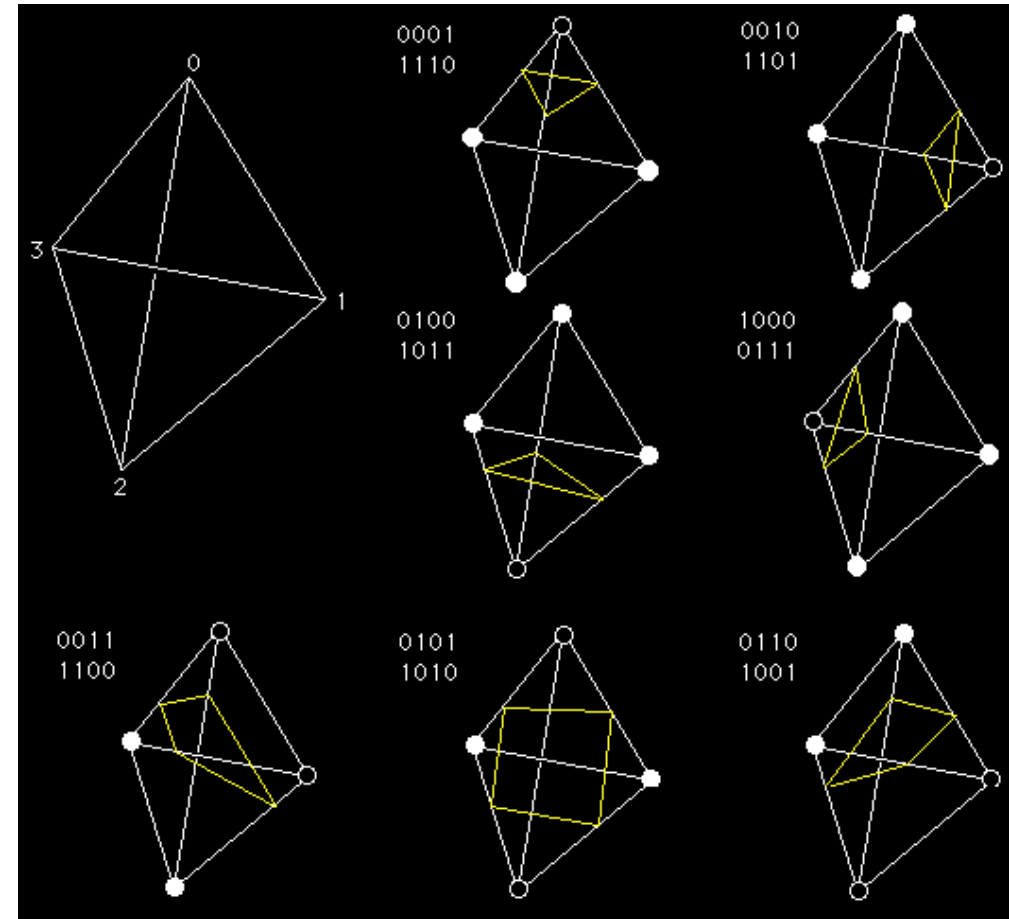
Marching Tetrahedra

- Marching Tetrahedra [Shirley et al. 1990]
- Primarily used for unstructured grids
- Also possible to split cells into tetrahedra



Marching Tetrahedra

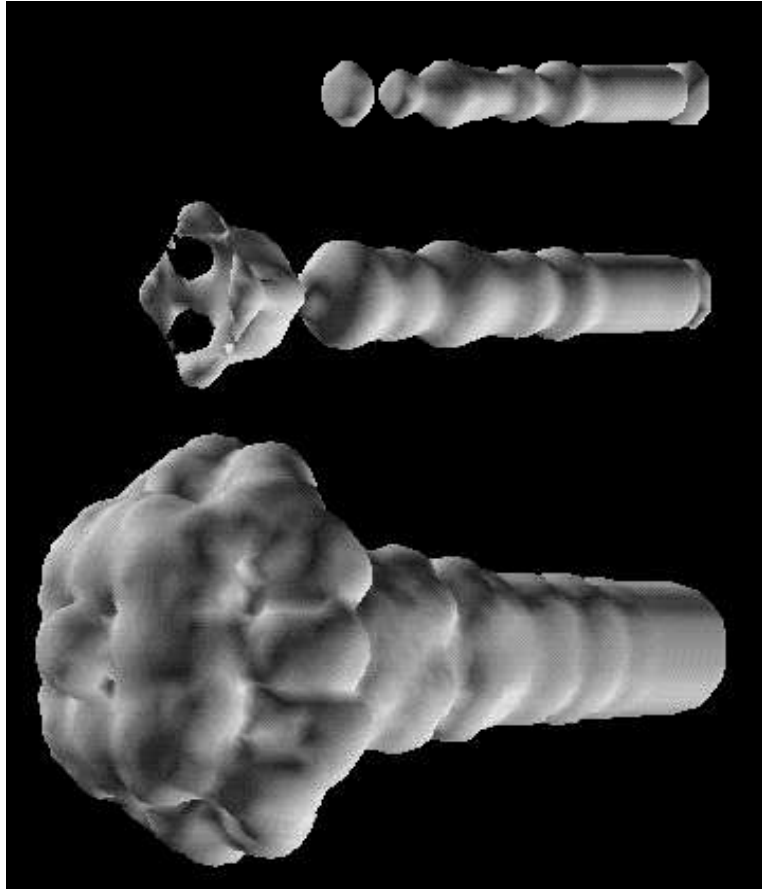
- Process each tetrahedron similarly to the Marching Cubes algorithm
 - Two different cases:
 - A) one – and three + (or vice versa)
 - The surface is defined by one triangle
 - B) two – and two +
 - Sectional surface given by a quadrilateral
 - Split it into two triangles using the shorter diagonal



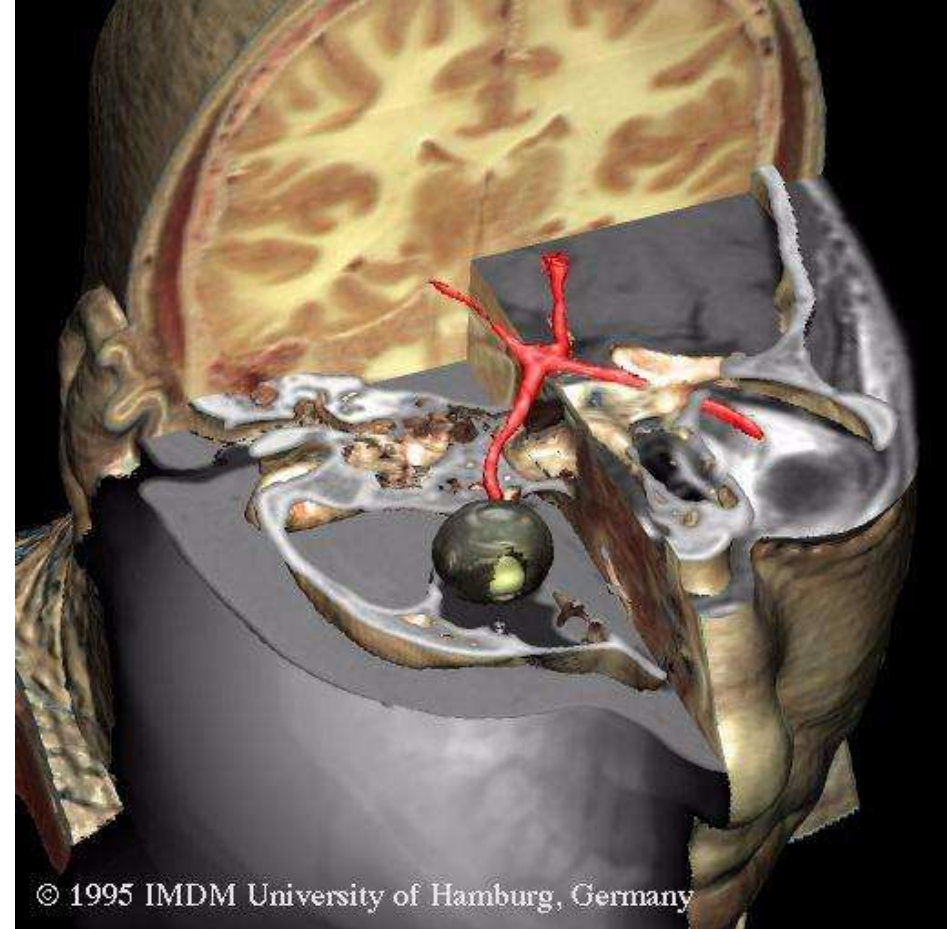
Properties of Marching Tetrahedra

- Fewer cases (3 instead of 15)
- No problems with consistency between adjacent cells
- **Quiz:** Why not create isosurfaces for hexahedral voxels by splitting them into tetrahedra?
 - *Answer 1:* Number of generated triangles might increase considerably compared to MC
 - *Answer 2:* Result not unique (depends on split)

Marching Cubes: Examples



Isosurface at different time steps



Isosurfaces in 3D medical data set

Summary: Isosurface Extraction

- **Marching Cubes** is the most widely used algorithm to extract isosurfaces from regular hexahedral grids
 - Processes each cell separately
 - Intersections based on trilinear interpolation
 - Triangulation based on case table
 - Asymptotic decider to resolve ambiguities
- **Marching Tetrahedra** are an alternative for unstructured grids

Section 9.5: Shading Isosurfaces

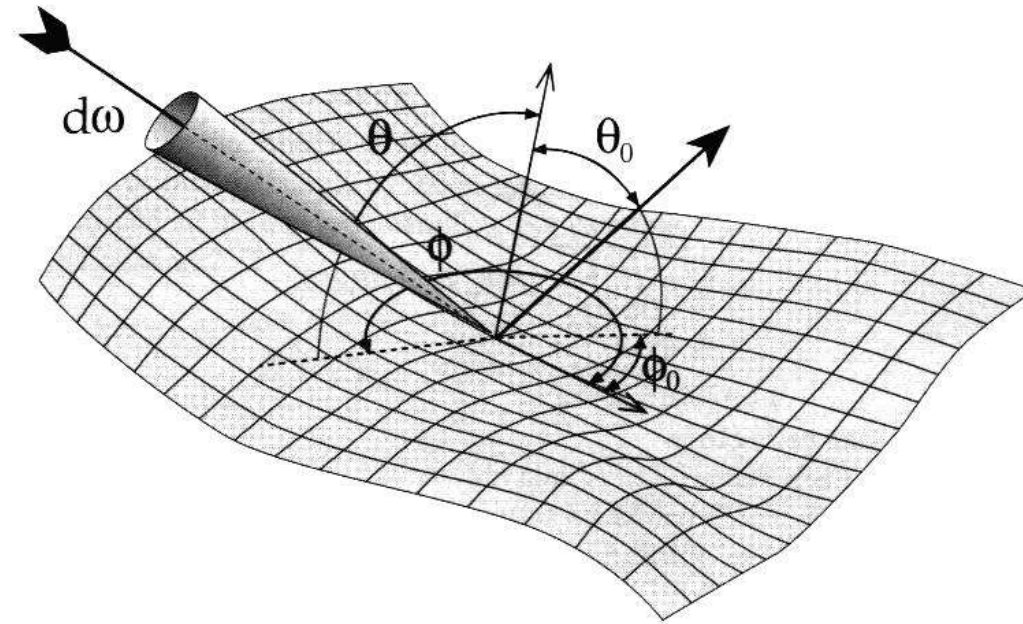
Surface Shading

- **Shading** is required to see the shape of the isosurface
- Place one or several **light sources** in the scene
 - *Often*: Point light or directional light
- Add light from all sources, reflected according to BRDF



BRDFs

- **Bidirectional Reflectance Distribution Function (BRDF)** captures how light is reflected. It depends on
 - Incoming light direction
 - Direction from which we observe the surface

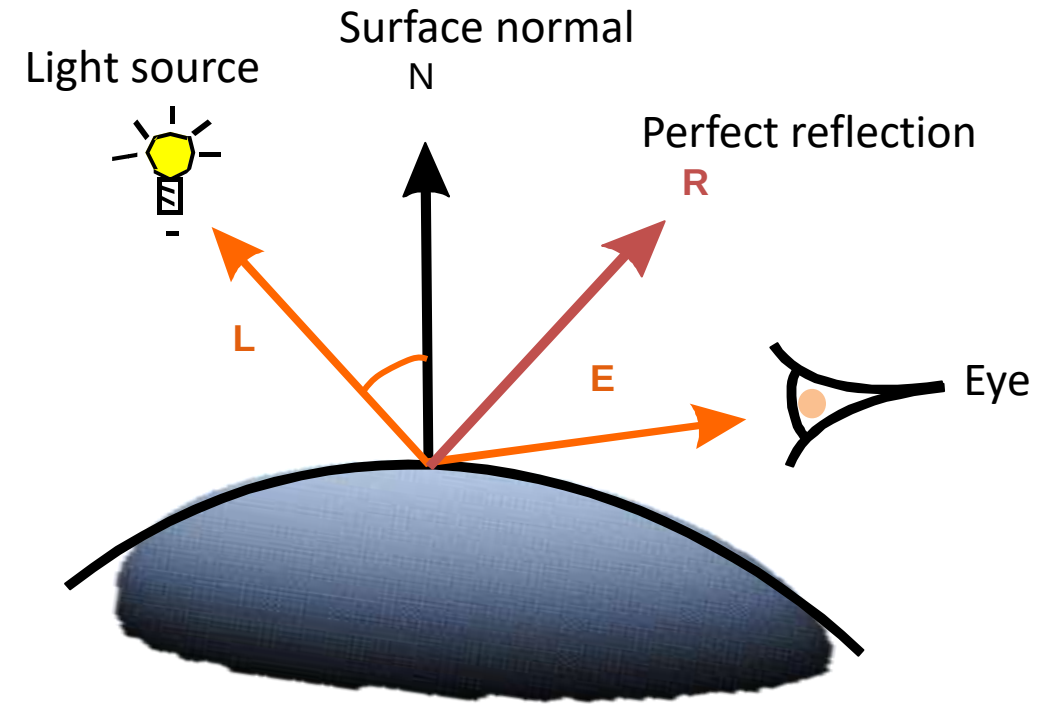
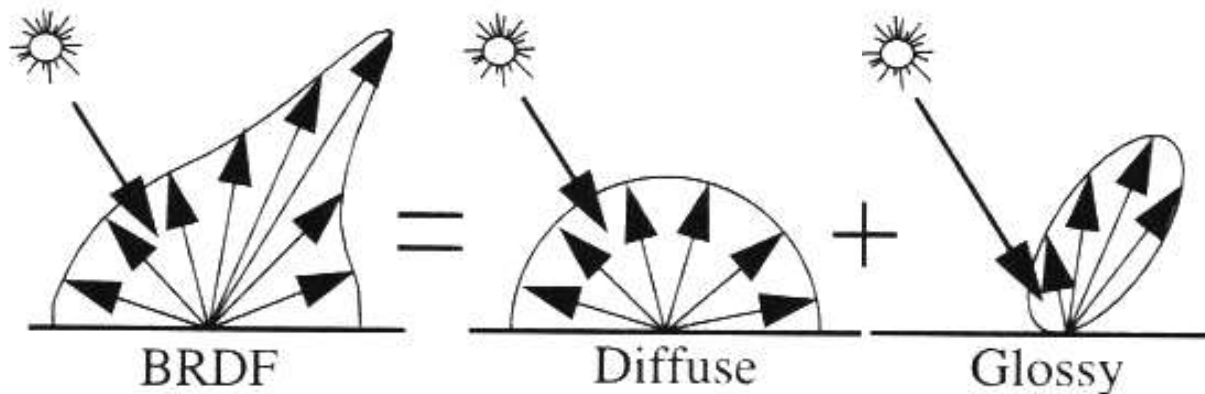


Phong Illumination Model

Phong Model: Simple and fast closed-form (nonphysical) BRDF

Terms for

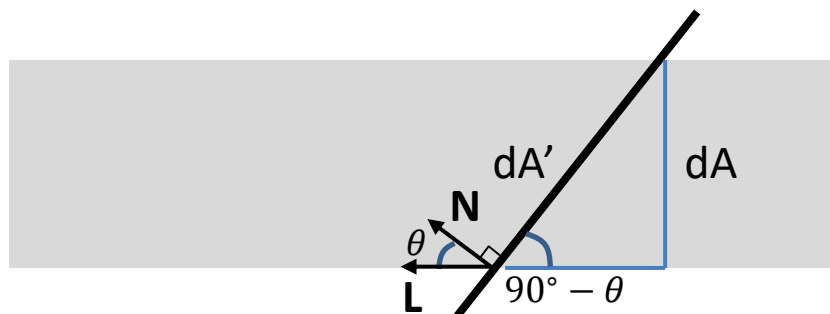
- Ambient light
- Diffuse light
- Specular light



Ambient / Diffuse Terms in Phong Model



- Ambient light: $C = k_a C_a O_d$
 - k_a is ambient contribution
 - C_a is color of ambient light
 - O_d is diffuse color of object
- Diffuse light: $C = k_a C_a O_d + k_d C_p O_d \cos(\theta)$
 - k_d is diffuse contribution
 - C_p is color of point/directional light
 - O_d is diffuse color of object
 - θ is angle between light and surface normal

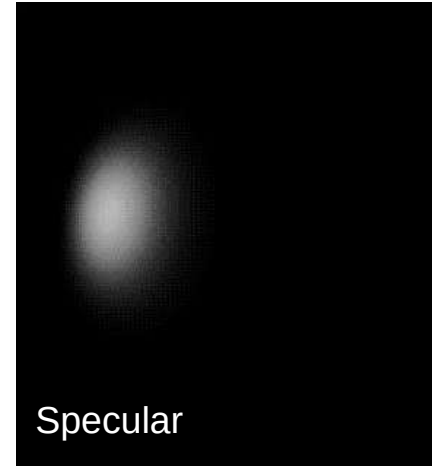


$$\frac{dA}{dA'} = \sin(90^\circ - \theta) = \cos(\theta)$$

Reminder: Irradiance = Radiant Flux / Area

Specular Term in Phong Model

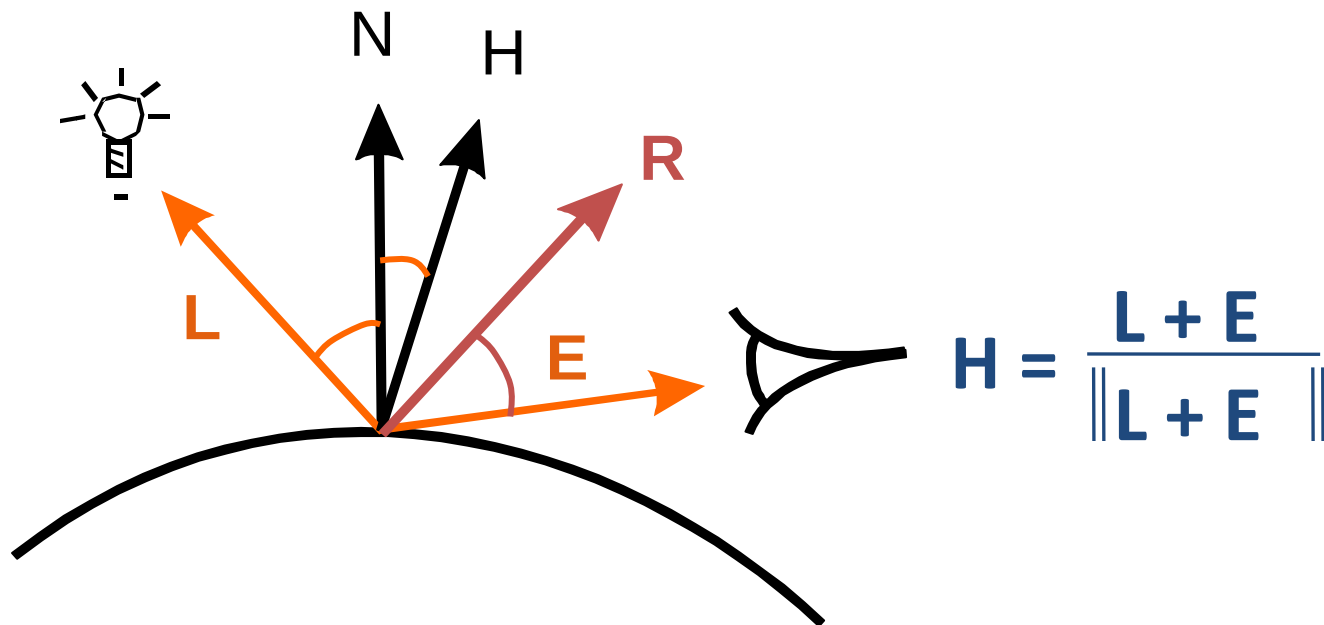
- Specular light: $C = k_a C_a O_d + k_d C_p O_d \cos(\theta) + k_s C_p O_s \cos^n(\sigma)$
 - k_s is specular contribution
 - C_p is color of point/directional light
 - O_s is specular color of object
 - n is exponent of highlight
 - σ is either
 - angle between reflected light and eye direction (Phong) or
 - angle between surface normal and halfway vector (Blinn-Phong; easier to compute)



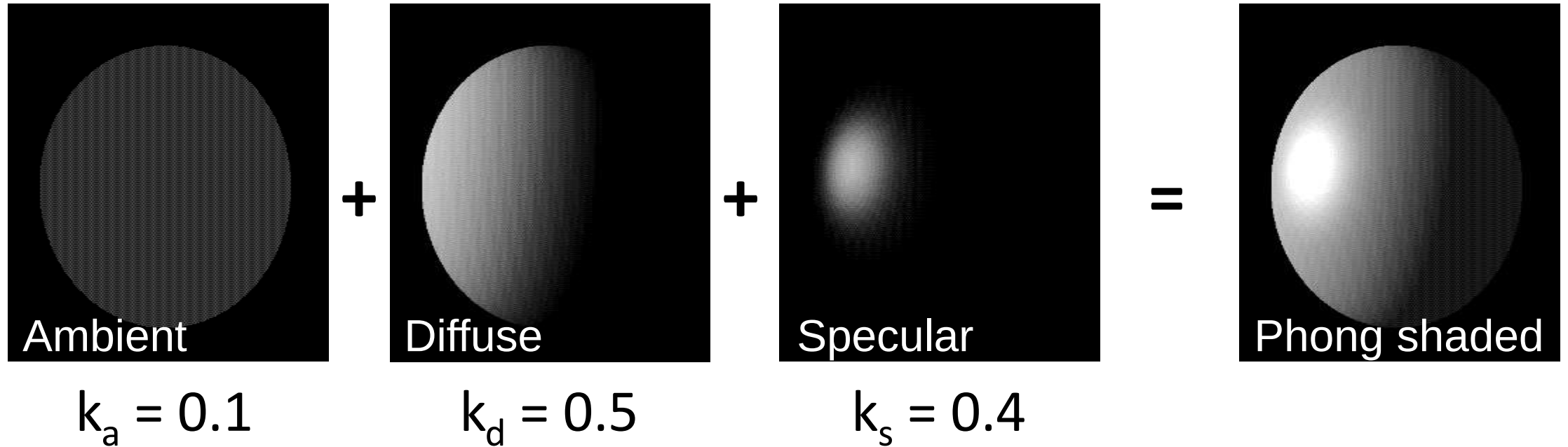
Specular Models

$$\cos(\sigma) = \mathbf{R} * \mathbf{E} \quad (\text{Phong})$$

$$\cos(\sigma) = \mathbf{N} * \mathbf{H} \quad (\text{Blinn-Phong})$$



Overall Effect of Phong Shading



Section 9.6:

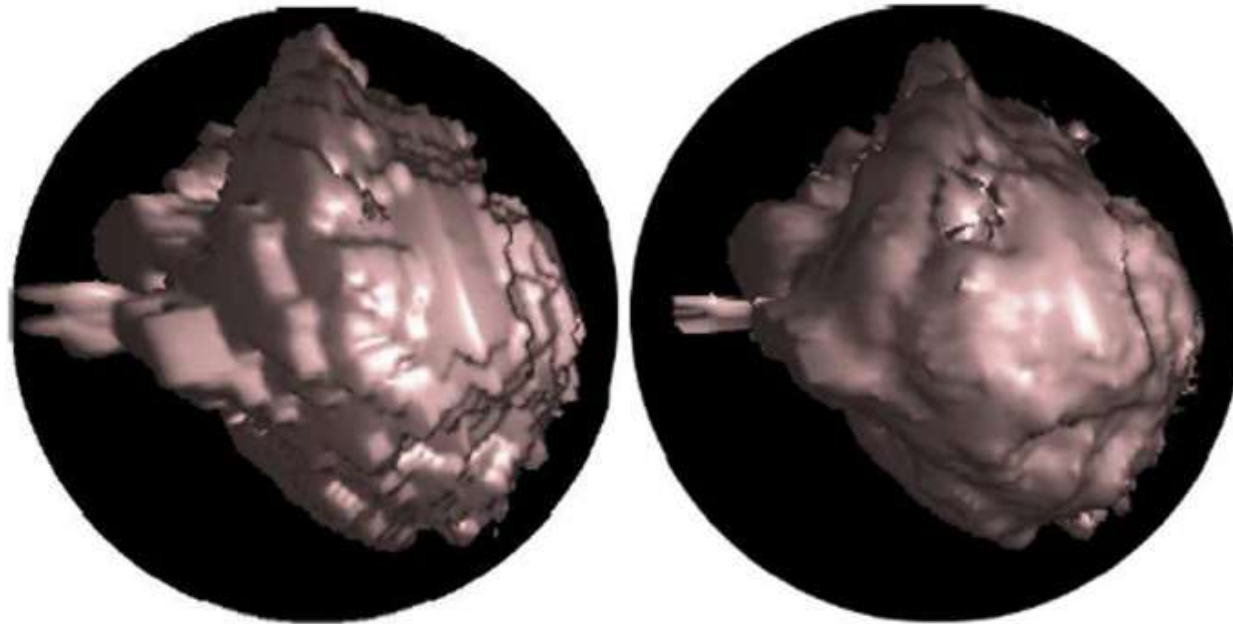
Post-Processing Isosurfaces

Motivation

- **Practical problems** with Marching Cubes:
 1. If underlying field is not smooth (e.g., binary segmentation result), resulting surface is “blocky”
 2. It produces a huge number of geometric primitives
- **Solutions** to these problems include:
 1. Mesh smoothing
 2. Mesh decimation

Mesh Smoothing

- Developed in Computer Graphics to remove noise from surface meshes
- Used in Visualization to post-process surfaces from noisy, highly quantized (e.g., binary) or highly anisotropic data

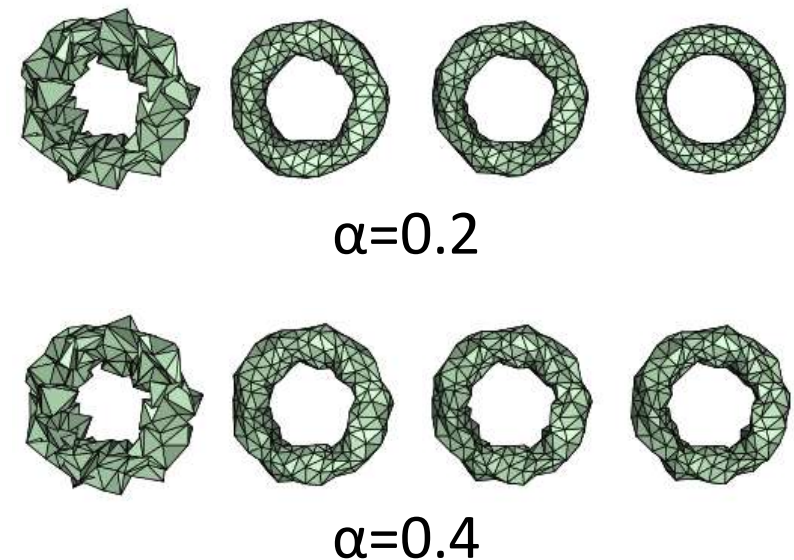
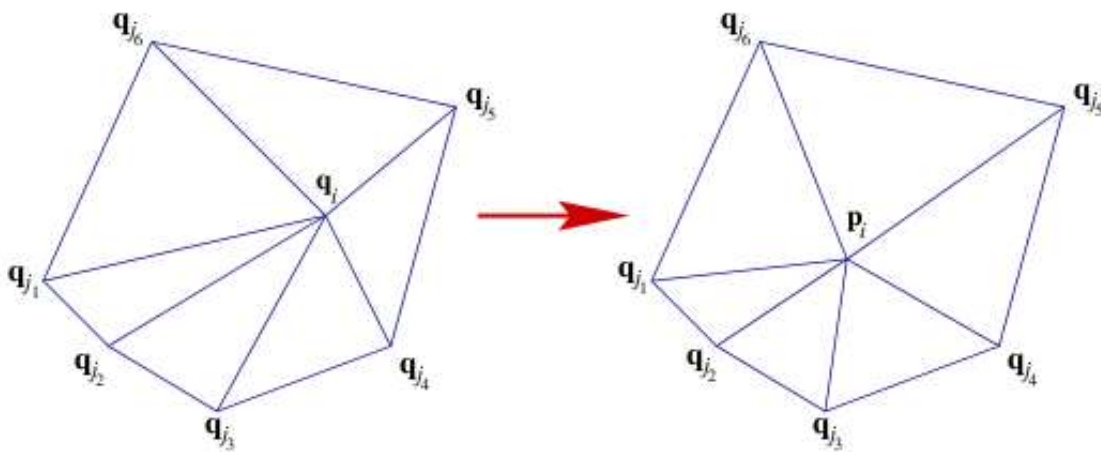


Extended Laplacian Smoothing

Example: [Vollmer et al. 1999] reduce shrinkage in Laplacian smoothing by keeping points anchored at original position $\mathbf{o}_i$ with weight $\alpha \in [0,1]$

$$\mathbf{p}_i := \begin{cases} \alpha \mathbf{o}_i + \frac{1-\alpha}{|adj(i)|} \sum_{j \in adj(i)} \mathbf{q}_j, & i \in V_{var}, \\ \mathbf{q}_i, & i \in V_{fix}. \end{cases}$$

$\mathbf{p}_i$ = updated point position $\mathbf{q}_i$ = point position in previous iteration



HC Correction Step

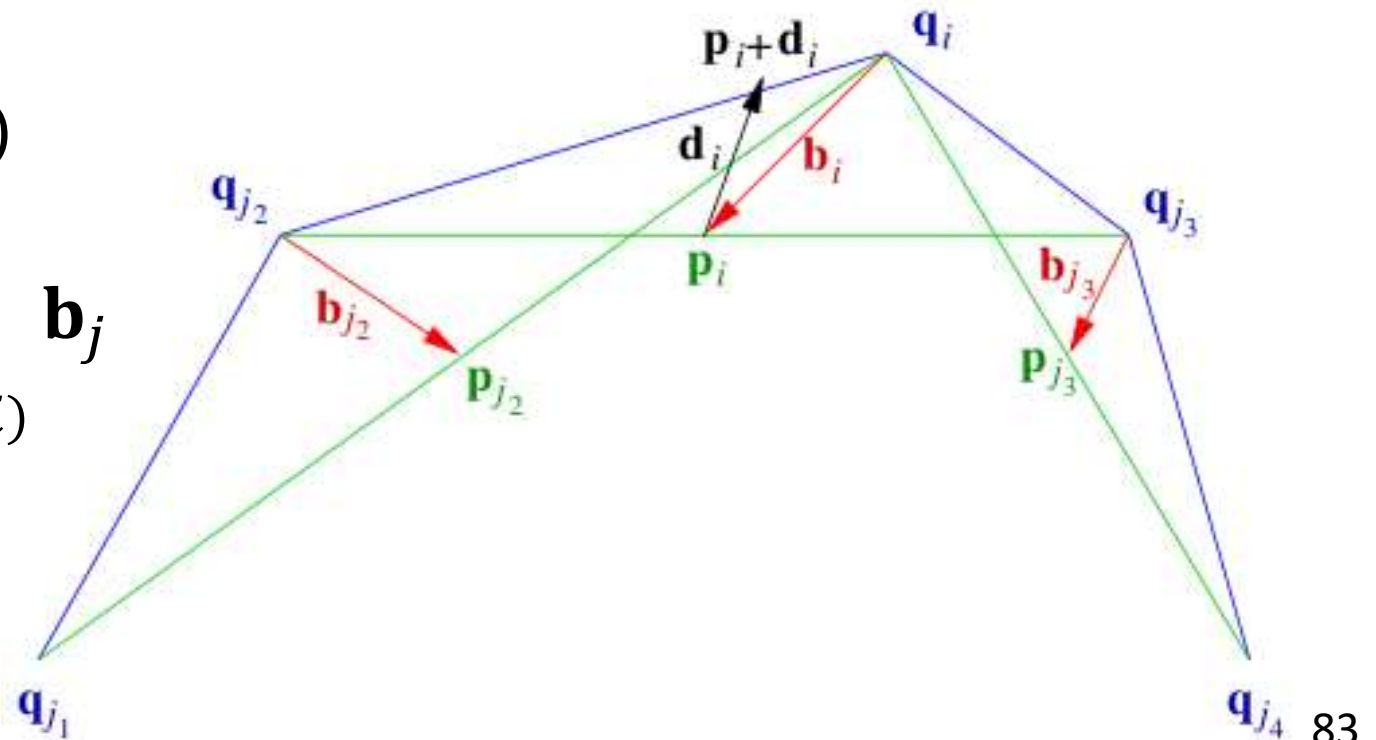
To further reduce shape deformations and shrinkage, the HC correction step moves vertices back towards the original surface

- Accounts for the displacement of the current point with weight $\beta \in [0,1]$, but also for its neighbors

$$\mathbf{b}_i := \mathbf{p}_i - (\alpha \mathbf{o}_i + (1 - \alpha) \mathbf{q}_i)$$

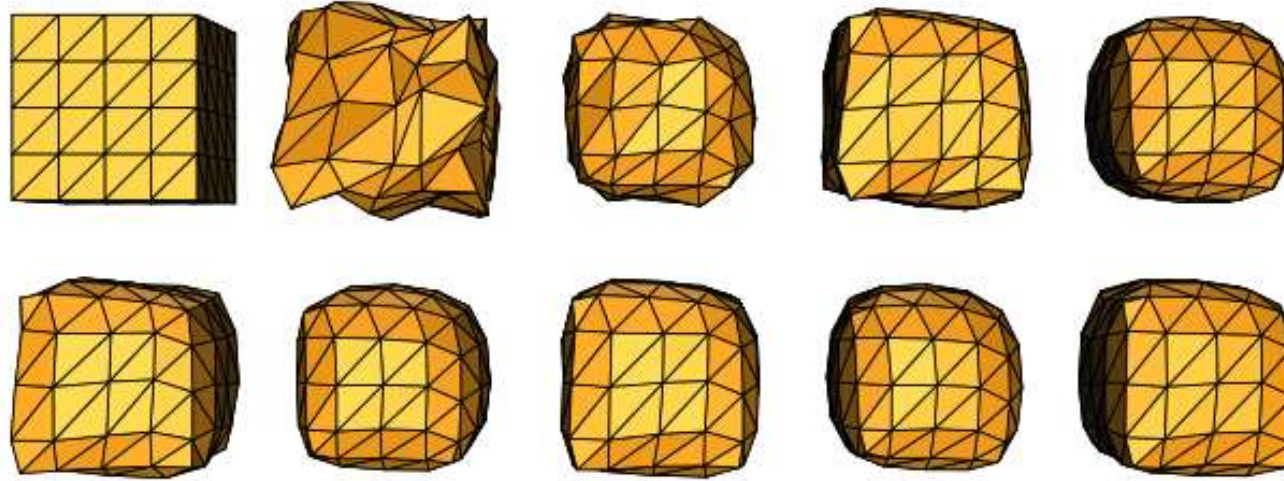
$$\mathbf{d}_i := -\beta \mathbf{b}_i - \frac{1 - \beta}{|\text{adj}(i)|} \sum_{j \in \text{adj}(i)} \mathbf{b}_j$$

Note: For simplicity,
sketch assumes $\alpha = 0$

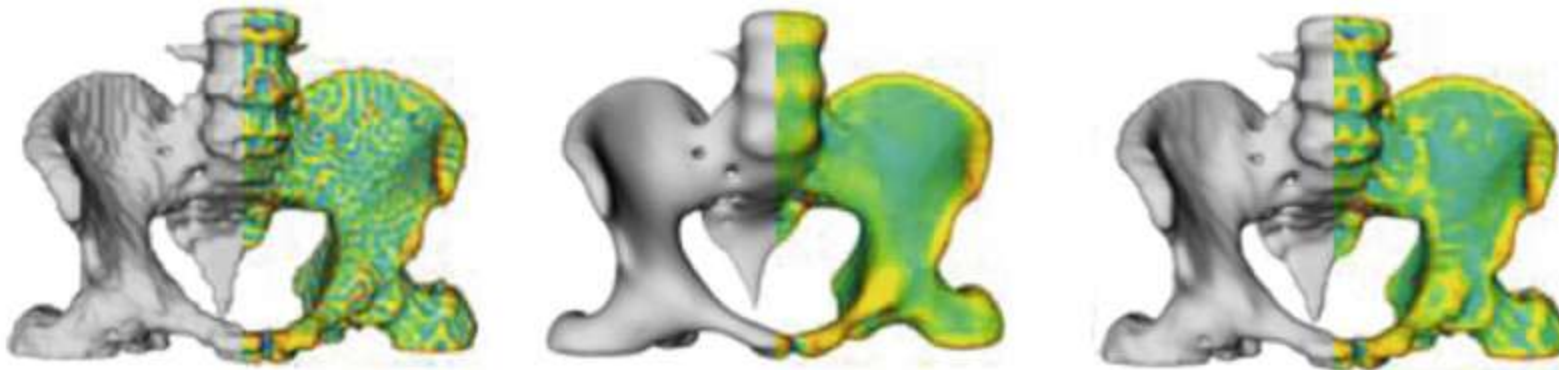


Extended Laplacian + HC: Examples

Alternate extended Laplacian smoothing and HC



Laplacian smoothing without and with correction



Motivation

- **Practical problems** with Marching Cubes:
 1. If underlying field is not smooth (e.g., binary segmentation result), resulting surface is “blocky”
 2. It produces a huge number of geometric primitives
- **Solutions** to these problems include:
 1. Mesh smoothing
 2. Mesh decimation

Mesh Decimation

- **Idea:** Remove vertices and re-triangulate with fewer triangles
 - Limit deviation between mesh before and after decimation
- *Optional:* Generate hierarchical mesh structure for level-of-detail rendering

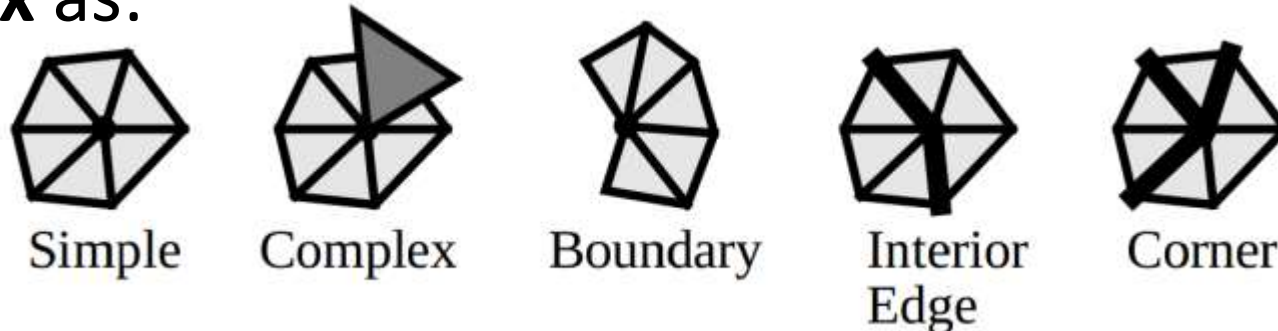


Decimation Method by Schroeder et al.

- Mesh decimation by [Schroeder et al. 1992]
 - **Goal:** Reduce number of triangles, but preserve topology and a good approximation of geometry
 - **Basic Algorithm:** Iterate over all vertices until desired number is reached or no more vertices can be removed:
 1. Classify the local geometry and topology for this given vertex
 2. Use the decimation criterion to decide if the vertex can be deleted
 3. If the point is deleted, re-triangulate the resulting hole

Decimation: Vertex Classification

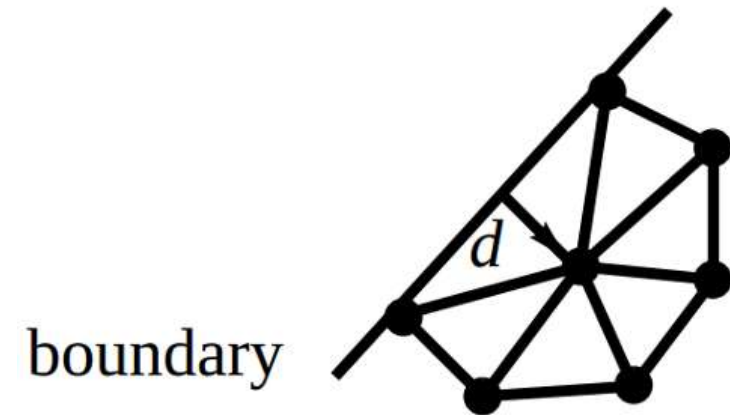
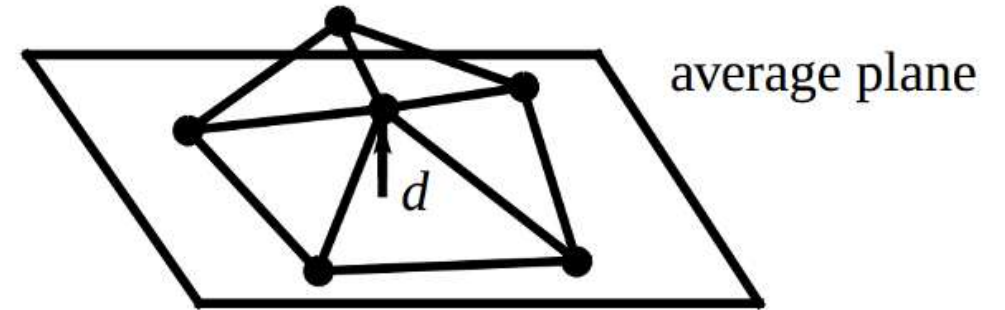
- Detect **feature edges**
 - Based on large angle between normals
 - Should be preserved during simplification
- **Classify vertex** as:



- **Simple** if surrounded by a complete cycle, each edge is used by exactly two triangles, no feature edges
 - **Boundary** if the cycle is not complete
- **Interior edge** and **corner** based on feature edges
- **Complex** and **corner vertices** will not be removed to preserve topology of mesh and feature edges

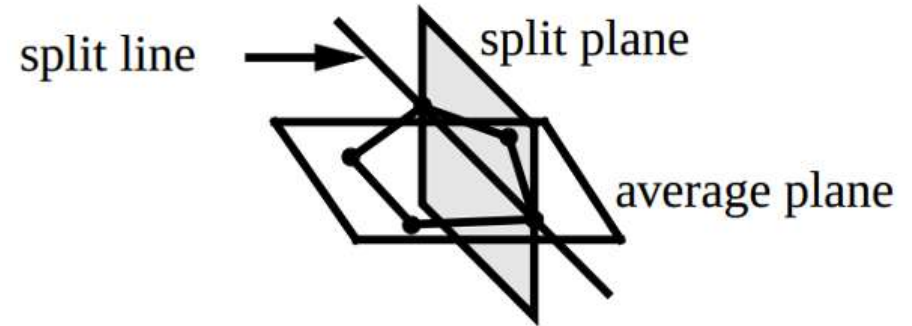
Decimation Criteria

- For **simple vertices**, compute distance d to average plane of neighbors to decide if it can be removed
 - Average plane formed by averaging center points and normals of incident triangles
- For **boundary and interior edge vertices**, compute distance d to line



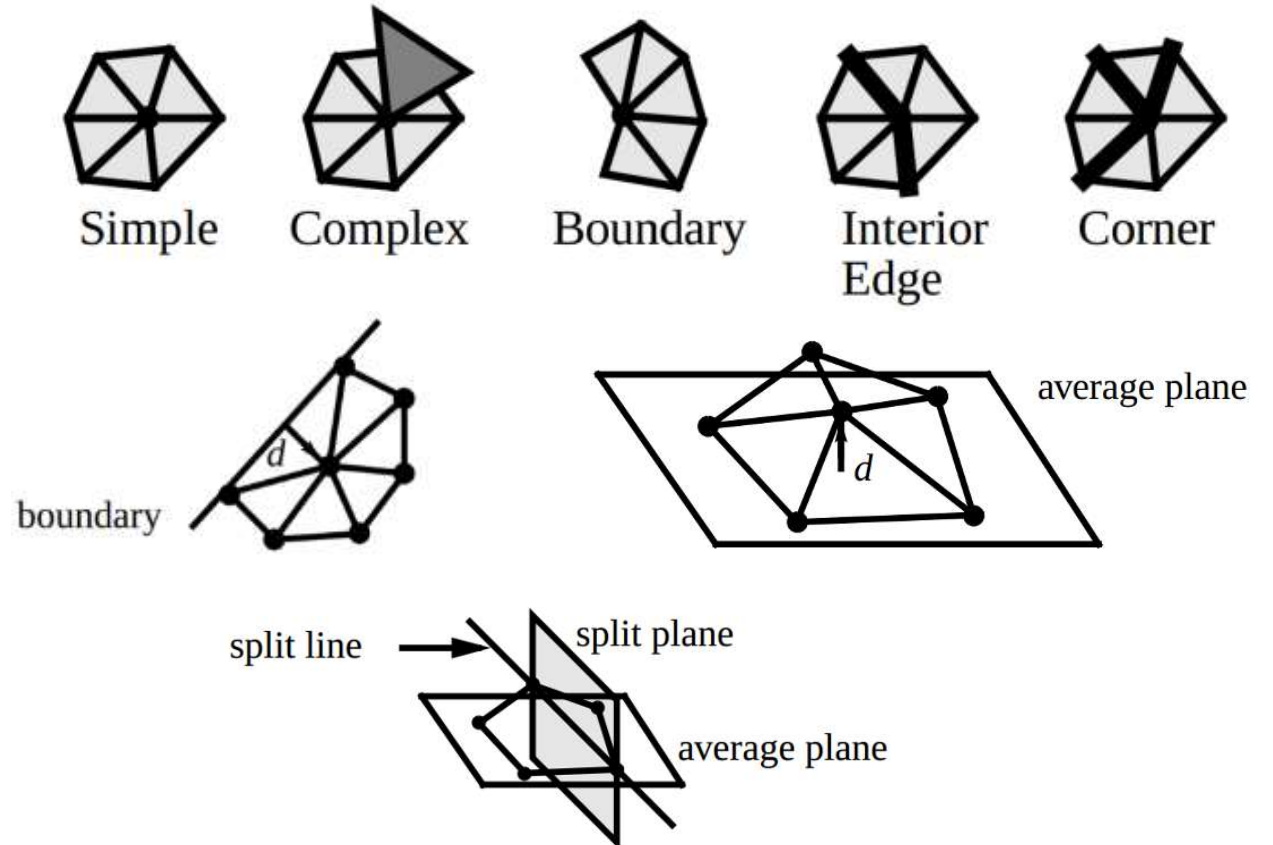
Triangulating The Hole

- Removing a vertex leaves a (generally non-planar) polygonal hole that needs to be triangulated
- Recursive splitting-based scheme
 - Reconstruct feature edges
 - Otherwise, find “split plane” that contains two non-adjacent vertices and is orthogonal to average plane
 - Split is valid if the two candidate loops fall entirely on opposite sides of the plane
 - Do not remove vertex if no valid split is found
 - If multiple splits are valid, perform the one that maximizes $(\text{minimum distance of loop vertices to plane}) / (\text{length of split line})$

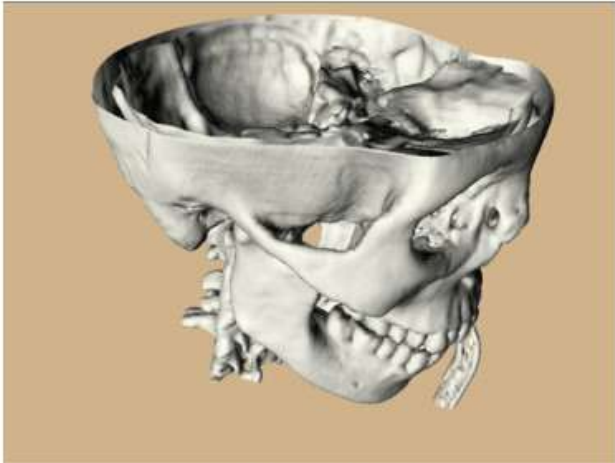


Summary: Decimation by Schroeder et al.

- Mesh decimation by [Schroeder et al. 1992]
 - **Basic Algorithm:** Iterate over all vertices until desired number is reached or no more vertices can be removed:
 1. Classify the local geometry and topology for this given vertex
 2. Use the decimation criterion to decide if the vertex can be deleted
 3. If the point is deleted, re-triangulate the resulting hole.



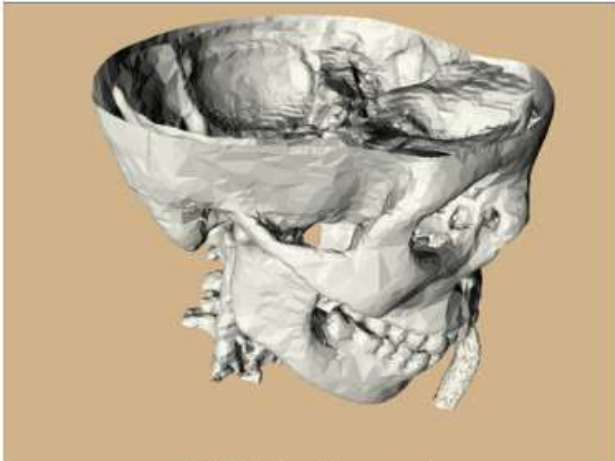
Mesh Decimation: Results



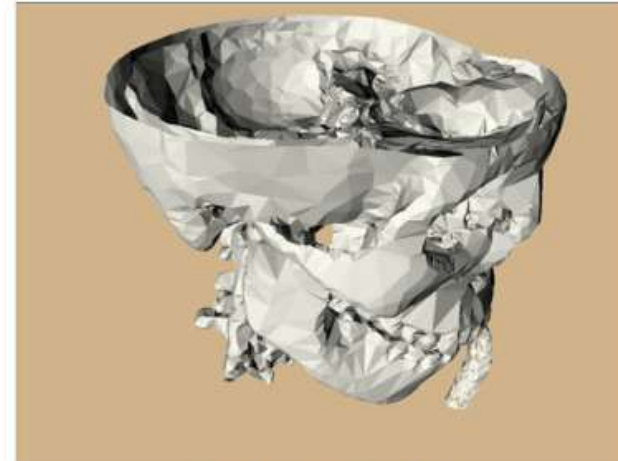
Full Resolution
(569K Gouraud shaded triangles)



75% decimated
(142K Gouraud shaded triangles)



75% decimated
(142K flat shaded triangles)



90% decimated
(57K flat shaded triangles)

References: Textbooks

- C.D. Hansen, C. Johnson, *Visualization Handbook*, Academic Press, 2004
- B. Preim, C. Botha: *Visual Computing for Medicine: Theory, Algorithms, and Applications*. Morgan Kaufmann, 2014

References: Research Papers

- W.E. Lorensen, H.E. Cline: *Marching cubes: A high resolution 3D surface construction algorithm*. Proc. ACM SIGGRAPH 1987, pp. 163-169
- G.M. Nielson, B. Hamann: *The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes*. IEEE Visualization 1991, pp. 83-93
- William J. Schroeder, Jonathan A. Zarge, William E. Lorensen: *Decimation of triangle meshes*. Proc. ACM SIGGRAPH 1992, pp. 65-70
- J. Vollmer, R. Mencl, H. Müller: *Improved Laplacian Smoothing of Noisy Surface Meshes*. Computer Graphics Forum 18(3):131-138, 1999

Exam-like Questions

1. How can Marching Squares / Marching Cubes implementations deal with cases in which the isovalue is exactly the same as the value at one of the given voxels?
2. State a benefit and a drawback of the Sobel operator compared to central differences.
3. Describe how an isosurface will look when rendered only with the ambient light term from the Phong model.
4. Why and in what way are meshes from the Marching Cubes algorithm often post-processed?